

Flight Board

Live Prague Airport departure board on Google Kubernetes Engine

Live: <https://flights.peterhudcovic.tech>

Code: <https://github.com/PeterHudcovic/flight-board>

Built in one night with two AI coding agents:

Codex owned the application, Claude Code owned the infrastructure.

Every change went through a pull request reviewed by the other agent and merged by me.

Screenshots taken on 10 October 2026 from the running system.

Secret values, keys and tokens are never shown.

PRAGUE AIRPORT (PRG) · DEPARTURES · TERMINAL 2

15:52:54

Sched.	Exp.	Destination	Flight	Bag Group	Check-in	Remark	Sched.	Exp.	Destination	Flight	Bag Group	Check-in	Remark	Sched.	Exp.	Destination	Flight	Bag Group	Check-in	Remark
14:55		MILAN BERGAMO	FR3529	209		Gate closed	16:05		WARSAW	LEC22				18:40		WARSAW	LEC26			
15:35		MADRID	IB810	212		Gate closed	16:05		SPLIT	OU4325				18:50		PORTO	U26858			
16:10		ROME CIAMPINO	FR6412	211		Boarding	16:20		RHODES	G31112				20:05		TREVISO	FR1530			
16:25		NAPOLI	FR5425	210		Boarding	16:30		VALENCIA	G31054				20:25		PARIS	AF1083			
16:40		MUNICH	LH1693	268-270			16:35		AMSTERDAM	KL1358				20:30		AMSTERDAM	KL1360			
16:50		VIENNA	OS648	268-270			16:40		FRANKFURT	LH1461				20:35		ZADAR	FR8555			
17:00		FRANKFURT	DE4408	234			16:00		ATHENS	A3867				20:40		BUDAPEST	FR4033			
17:20		MÁLAGA	G31152	238-238			16:25		KOS	G31128				20:50		BARCELONA	VY8555			
17:40		GDAŃSK	FR1821				16:30		HELSINKI	AY1226				20:55		COPENHAGEN	SK7766			
18:00		THESSALONIKI	A3517				16:30		ZURICH	LX1469				21:10		ALICANTE	U25552			
18:00		PALMA DE MÁLAGA	G31164				16:35		RIGA	BT482				21:20		LUCA	KM539			
18:00		PIEMONTE ISLAND	G31168				16:40		MADRID	IB812				21:40		VILNIUS	W618...			

Last update 15:47 [data:airportDataBox](#)

Overview

0. Architecture	4
-----------------	---

The product

1. Live departure board	5
-------------------------	---

Kubernetes platform

2. GKE cluster	6
3. Node pool and nodes	7
4. Workloads	8
5. First rollout in k9s	9
6. Backend deployment: replicas and image	10
7. Backend pods across nodes	11
8. Frontend deployment: replicas and image	12
9. Frontend pods across nodes	13
10. Backend YAML: metadata and replicas	14
11. Backend YAML: update strategy	15
12. Backend YAML: secrets, config and liveness probe	16
13. Backend YAML: ports and readiness probe	17
14. Backend YAML: resources, security and startup probe	18
15. Backend YAML: pod security and topology spread	19
16. Backend YAML: status	20
17. Frontend YAML: image and liveness probe	21
18. Frontend YAML: readiness, resources and security	22
19. Frontend YAML: pod security and topology spread	23
20. Frontend YAML: status	24
21. PodDisruptionBudgets	25
22. PDBs for backend and frontend	26
23. PDB for MongoDB	27

Application configuration

24. ConfigMap flight-board-config	28
25. Secrets (names only)	29

Request path

26. Services	30
27. Ingress	31
28. Load balancer	32
29. Load balancer frontend	33
30. TLS certificate	34
31. Host and path rules	35
32. Backend service for the API	36
33. Backend service for the frontend	37

34. Health checks	38
35. Network endpoint groups	39

Network

36. Private nodes	40
37. VPC and subnet	41
38. Cloud NAT	42
39. Firewall rules	43

Data

40. MongoDB storage	44
41. MongoDB resource: placement and resources	45
42. MongoDB resource: users and status	46
43. MongoDB replica set members	47

CI/CD

44. Container images	48
45. GitHub Actions	49
46. Deploy runs	50
47. Deploy run #3	51
48. Deploy run #3: steps	52
49. Deploy run #3: version check	53
50. Two agents in parallel worktrees	54

Infrastructure as code

51. Terraform state	55
---------------------	----

Security

52. Service accounts without keys	56
53. Project roles of the service accounts	57
54. Where project roles are set (Terraform)	58
55. Resource-level roles (Terraform)	59
56. Node pool service account (live)	60
57. Node pool scope (Terraform)	61
58. Workload Identity Federation	62
59. Provider attribute mapping	63
60. Service accounts bound to the pool	64
61. WIF condition and mapping (Terraform)	65
62. WIF bindings to service accounts (Terraform)	66
63. Deploy workflow identity	67
64. Terraform plan workflow identity	68
65. Kubernetes RBAC Role for deploy	69
66. Kubernetes RBAC binding for deploy	70

Edge

- 67. Cloudflare DNS record 71
- 68. Cloudflare SSL/TLS mode 72
- 69. Cloudflare Configuration Rules 73
- 70. Rule flights strict SSL: condition and setting 74

Cost

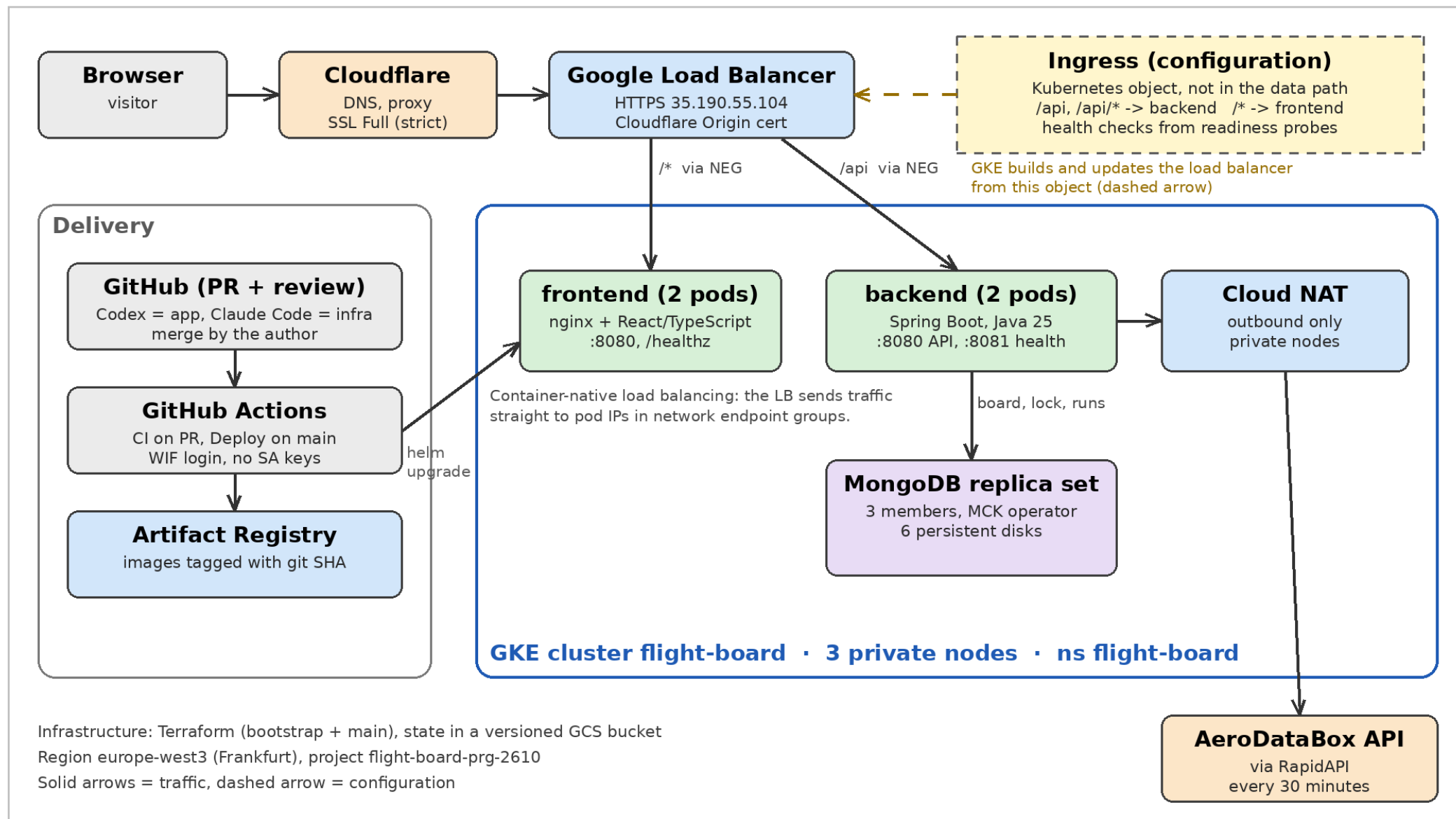
- 71. Free trial credit and month to date 75
- 72. Cost by service 76
- 73. Budget 77

Delivery process

- 74. Repository 78
- 75. Pull requests 79
- 76. Cross-review between agents 80

Operations

- 77. Logs 81



A request goes from the browser through Cloudflare to the Google load balancer, which sends it straight to frontend or backend pod IPs in network endpoint groups. The Ingress is not in that path: it is the configuration from which GKE builds the load balancer. The backend stores the board in MongoDB and fetches new data through Cloud NAT.

1. Live departure board

The product

PRAGUE AIRPORT (PRG) · DEPARTURES · TERMINAL 2

15:52:54

Sched.	Exp.	Destination	Flight	Bag Drop	Check-in	Remark	Sched.	Exp.	Destination	Flight	Bag Drop	Check-in	Remark	Sched.	Exp.	Destination	Flight	Bag Drop	Check-in	Remark
14:55		MILAN BERGAMO	FR3529		209	Gate closed	18:05		WARSAW	L0522				19:40		WARSAW	L0526			
15:35		MADRID	IB810		212	Gate closed	18:05		SPLIT	OU4325				19:50		PORTO	U26858			
16:10		ROME CIAMPINO	FR8412		211	Boarding	18:20		RHODES	QS1112				20:05		TREVISO	FR1530			
16:25		NÁPOLI	FR5425		210	Boarding	18:30		VALENCIA	QS1054				20:25		PARIS	AF1083			
16:40		MUNICH	LH1693		268-270		18:35		AMSTERDAM	KL1358				20:30		AMSTERDAM	KL1360			
16:50		VIENNA	OS646		268-270		18:40		FRANKFURT	LH1401				20:35		ZADAR	FR8555			
17:00		FRANKFURT	DE4408		234		19:00		ATHENS	A3867				20:40		BUDAPEST	FR4033			
17:20		MÁLAGA	QS1152		236-238		19:25		KOS	QS1128				20:50		BARCELONA	VY8655			
17:40		GDAŃSK	FR1921				19:30		HELSINKI	AY1226				20:55		COPENHAGEN	SK1766			
18:00		THESSALONIKI	A3517				19:30		ZURICH	LX1489				21:10		ALICANTE	U25552			
18:00		PALMA DE MALLORCA	QS1164				19:35		RIGA	BT482				21:20		LUQA	KM539			
18:00		MENORCA ISLAND	QS1168				19:40		MADRID	IB812				21:40		VILNIUS	W619...			

Last update 15:47 [data: AeroDataBox](#)

The live board at flights.peterhudcovic.tech shows real Terminal 2 departures from Prague, refreshed from AeroDataBox every 30 minutes, with gate status colours and the last update time.

The screenshot displays the Google Cloud console interface for a Kubernetes Engine cluster named 'flight-board'. The left sidebar shows the navigation menu with 'Kubernetes Engine' selected. The main content area shows the 'Cluster details' page for 'flight-board', which is in a 'Ready' state (indicated by a green checkmark). The 'Details' tab is active, showing a table of cluster basics.

Cluster basics		
Name	flight-board	🔒
Mode	Standard	🔒
Autopilot compute class compatibility	Enabled	ⓘ
Location type	Zonal	🔒
Control plane zone	europa-west3-a	🔒
Default node zones	europa-west3-a	✎
Total size	3	ⓘ
Release channel	Regular channel	✎
Version	1.35.8-gke.1225000	✎ Upgrade available
Current COS version	cos-125-19216-532-135	ⓘ
End of standard support	Apr 11, 2027	ⓘ
End of extended support	Feb 11, 2028	ⓘ
Rollout sequence	To use rollout sequencing, register your cluster to a fleet	🔒

Below the table, the 'Upgrades' section is visible.

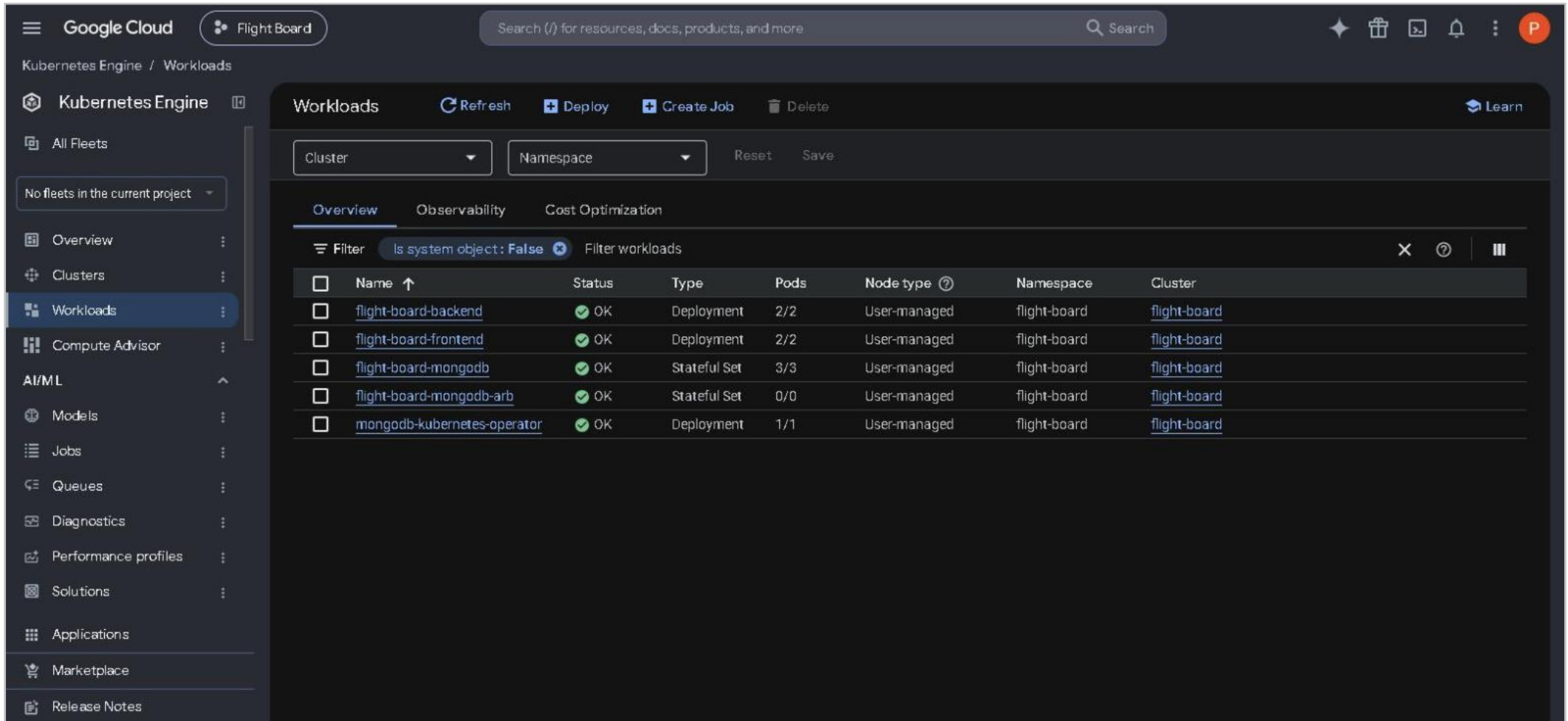
A zonal GKE Standard cluster in europa-west3-a (Frankfurt) on the Regular release channel, created entirely by Terraform.

The screenshot shows the Google Cloud console interface for a Kubernetes cluster named 'flight-board'. The left sidebar contains navigation links for 'Kubernetes Engine', 'All Fleets', 'Overview', 'Clusters', 'Workloads', 'Compute Advisor', 'AI/ML', 'Models', 'Jobs', 'Queues', 'Diagnostics', 'Performance profiles', 'Solutions', 'Applications', 'Marketplace', and 'Release Notes'. The main content area displays the 'flight-board' cluster details, with the 'Nodes' tab selected. The 'Node pools' section shows a single node pool named 'flight-board-nodes' with a status of 'Ok'. The node pool is user-managed, has 3 nodes, and is running Container-Optimized OS. Below the node pool table, the 'Nodes' section shows a single node 'gke-flight-board-nodes-5d1f78aa-' with a status of 'Ready'.

Name	Status	Version	Node pool type	Number of nodes	Machine type	Image type	Autoscaling	Default IPv4 P
flight-board-nodes	Ok	1.35.8-gke.1225000	User-managed	3	e2-standard-2	Container-Optimized OS with containerd (cos_containerd)	Off	10.20.0.0/16

Name	Status	Node type	CPU requested	CPU allocatable	Memory requested	Memory allocatable	Storage requi
gke-flight-board-nodes-5d1f78aa-	Ready	User-managed	1.5 CPU	1.93 CPU	2.04 GB	6.32 GB	

One node pool with three e2-standard-2 nodes running Container-Optimized OS; autoscaling is off to keep the cost predictable.



The screenshot displays the Google Cloud Kubernetes Engine Workloads page. The left sidebar shows the navigation menu with 'Workloads' selected. The main content area shows a table of workloads in the 'flight-board' namespace. All workloads are in a healthy state (OK).

Name	Status	Type	Pods	Node type	Namespace	Cluster
flight-board-backend	OK	Deployment	2/2	User-managed	flight-board	flight-board
flight-board-frontend	OK	Deployment	2/2	User-managed	flight-board	flight-board
flight-board-mongodb	OK	Stateful Set	3/3	User-managed	flight-board	flight-board
flight-board-mongodb-arb	OK	Stateful Set	0/0	User-managed	flight-board	flight-board
mongodb-kubernetes-operator	OK	Deployment	1/1	User-managed	flight-board	flight-board

All workloads are healthy: backend and frontend with 2 replicas each, the 3-member MongoDB StatefulSet and the MongoDB operator.

```

Context: gke_flight-board-prg-2610_europe-west3-a_flight-board [RW]
Cluster: gke_flight-board-prg-2610_europe-west3-a_flight-board
User: gke_flight-board-prg-2610_europe-west3-a_flight-board
K9s Rev: v0.51.0
K8s Rev: v1.35.8-gke.1225000
CPU: 8%
MEM: 30%

<0> all          <a> Attach      <ctrl-k> Kill      <o> Show Node
<1> flight-board <ctrl-d> Delete    <l> Logs      <f> Show PortForward
<2> default      <d> Describe <p> Logs Previous <t> Transfer
                  <e> Edit     <shift-f> Port-Forward <w> Warp To Namespace
                  <?> Help    <z> Sanitize  <y> YAML
                  <shift-j> Jump Owner <s> Shell

pods(flight-board) [8]

```

NAME↑	PF	READY	STATUS	RESTARTS	CPU	%CPU/R	%CPU/L	MEM	%MEM/R	%MEM/L	IP	NODE	AGE
flight-board-backend-64cb66b8fb-db562	●	0/1	ContainerCreatingΔ	0	0	0	n/a	0	0	0	n/a	gke-flight-board-flight-board-nodes-5d1f78aa-xgl4	2s
flight-board-backend-64cb66b8fb-x5qhx	●	0/1	ContainerCreating	0	0	0	n/a	0	0	0	n/a	gke-flight-board-flight-board-nodes-5d1f78aa-w5hc	2s
flight-board-frontend-84474c8d7b-mbvdp	●	0/1	ContainerCreatingΔ	0	0	0	n/a	0	0	0	n/a	gke-flight-board-flight-board-nodes-5d1f78aa-w5hc	2s
flight-board-frontend-84474c8d7b-t7n6m	●	0/1	ContainerCreatingΔ	0	0	0	n/a	0	0	0	n/a	gke-flight-board-flight-board-nodes-5d1f78aa-xgl4	2s
flight-board-mongodb-0	●	2/2	Running	0	64	8	6	177	19	11	10.20.2.4	gke-flight-board-flight-board-nodes-5d1f78aa-xgl4	33m
flight-board-mongodb-1	●	2/2	Running	0	65	8	6	166	18	11	10.20.0.4	gke-flight-board-flight-board-nodes-5d1f78aa-w5hc	31m
flight-board-mongodb-2	●	2/2	Running	0	69	9	6	163	18	10	10.20.1.10	gke-flight-board-flight-board-nodes-5d1f78aa-hxdp	29m
mongodb-kubernetes-operator-799c9f49dc-pqkkk	●	1/1	Running	0	1	1	0	16	8	3	10.20.2.3	gke-flight-board-flight-board-nodes-5d1f78aa-xgl4	35m

k9s view of namespace flight-board while the application was first deployed: two backend and two frontend pods 2 s old in ContainerCreating, already spread across different nodes, while the three MongoDB members (29 to 33 min old, 2/2 Running) and the MongoDB operator were already up. The ReplicaSet hashes differ from today's because later deploys created new revisions.

Labels

app.kubernetes.io/managed-by: Helm

app.kubernetes.io/name: flight-board-backend

app.kubernetes.io/part-of: flight-board

helm.sh/chart: flight-board-0.1.0

Node type ?

User-managed

Logs ?

[Container logs](#), [Audit logs](#)

Replicas

2 updated, 2 ready, 2 available, 0 unavailable

Pod specification

Revision 3, containers: [backend](#), volumes: [tmp](#)

Horizontal Pod Autoscaler ?

Not configured

Configure

Vertical Pod Autoscaler ?

Not configured

Configure

Active revisions

Revision	Name	Status	Summary	Created on	Pods running/Pods total
<u>3</u>	flight-board-backend-74fdf7648b	OK	backend: europe-west3-docker.pkg.dev/flight-board-prg-2610/flight-board/backend:8812e08538ae28974a856abbac3808fa93aa3f5c	Oct 10, 2026, 6:50:44 AM	2/2

flight-board-backend: 2 updated, 2 ready, 2 available, 0 unavailable. Active revision 3 runs image europe-west3-docker.pkg.dev/flight-board-prg-2610/flight-board/backend:8812e08538ae28974a856abbac3808fa93aa3f5c (tag = git commit SHA). No Horizontal or Vertical Pod Autoscaler is configured.

Managed pods

 Filter

?

☰

Revision	Name	Status	Restarts	Created on ↑	Node
3	flight-board-backend-74fdf7648b-jpwlg	✔ Running	0	Oct 10, 2026, 6:50:44 AM	gke-flight-board-flight-board-nodes-5d1f78aa-w5hc
3	flight-board-backend-74fdf7648b-gq9qn	✔ Running	0	Oct 10, 2026, 6:51:09 AM	gke-flight-board-flight-board-nodes-5d1f78aa-xgl4

Exposing services ?

Name ↑	Type	Endpoints
flight-board	Ingress	flights.peterhudcovic.tech/api ... ▼
flight-board-backend	Cluster IP	10.30.3.122

Configuration ?

Name	Type
flight-board-aerodatabox	🔑 Secret
flight-board-mongodb-admin-flightboard-app	🔑 Secret

The two backend pods run on different nodes (...-w5hc and ...-xgl4), 0 restarts. The deployment is exposed by ClusterIP service flight-board-backend (10.30.3.122) and the Ingress, and uses two Secrets: flight-board-aerodatabox and flight-board-mongodb-admin-flightboard-app.

Node type ?	User-managed
Logs ?	Container logs , Audit logs
Replicas	2 updated, 2 ready, 2 available, 0 unavailable
Pod specification	Revision 3, containers: frontend , volumes: tmp
Horizontal Pod Autoscaler ?	⊖ Not configured Configure
Vertical Pod Autoscaler ?	⊖ Not configured Configure

Active revisions					
↓ Revision	Name	Status	Summary	Created on	Pods running/Pods total
3	flight-board-frontend-8566b9795	✓ OK	frontend: europe-west3-docker.pkg.dev/flight-board-prg-2610/flight-board/frontend:8812e08538ae28974a856abbac3808fa93aa3f5c	Oct 10, 2026, 6:50:46 AM	2/2

flight-board-frontend: 2 updated, 2 ready, 2 available, 0 unavailable. Active revision 3 runs image .../flight-board/frontend:8812e08538ae28974a856abbac3808fa93aa3f5c, the same commit as the backend. No Horizontal or Vertical Pod Autoscaler is configured.

Active revisions

↓ Revision	Name	Status	Summary	Created on	Pods running/Pods total
3	flight-board-frontend-8566b9795	✓ OK	frontend: europe-west3-docker.pkg.dev/flight-board-prg-2610/flight-board/frontend:8812e08538ae28974a856abbac3808fa93aa3f5c	Oct 10, 2026, 6:50:46 AM	2/2

Managed pods

Filter pods



Revision	Name	Status	Restarts	Created on ↑	Node
3	flight-board-frontend-8566b9795-ms2tn	✓ Running	0	Oct 10, 2026, 6:50:46 AM	gke-flight-board-flight-board-nodes-5d1f78aa-xgl4
3	flight-board-frontend-8566b9795-zgrlw	✓ Running	0	Oct 10, 2026, 6:50:55 AM	gke-flight-board-flight-board-nodes-5d1f78aa-w5hc

Exposing services ?

Name ↑	Type	Endpoints
flight-board	Ingress	flights.peterhudcovic.tech/api ... ▼
flight-board-frontend	Cluster IP	10.30.10.133

The two frontend pods also run on different nodes (...-xgl4 and ...-w5hc), 0 restarts. Service flight-board-frontend is a ClusterIP (10.30.10.133) behind the Ingress. The Node column was switched on in the console view only.

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    annotations:
5      deployment.kubernetes.io/revision: '3'
6      meta.helm.sh/release-name: flight-board
7      meta.helm.sh/release-namespace: flight-board
8    creationTimestamp: '2026-10-10T01:46:48Z'
9    generation: 3
10   labels:
11     app.kubernetes.io/managed-by: Helm
12     app.kubernetes.io/name: flight-board-backend
13     app.kubernetes.io/part-of: flight-board
14     helm.sh/chart: flight-board-0.1.0
15   name: flight-board-backend
16   namespace: flight-board
17   resourceVersion: '1791607894516031024'
18   uid: 685a7516-4ca9-4529-a552-8c9b3a37beb0
19 spec:
20   progressDeadlineSeconds: 600
21   replicas: 2
22   revisionHistoryLimit: 10
23   selector:
24     matchLabels:
```

The Deployment is managed by Helm (chart flight-board-0.1.0, release flight-board, revision 3) with replicas: 2, progressDeadlineSeconds 600 and revisionHistoryLimit 10.

```
23 selector:
24   matchLabels:
25     app.kubernetes.io/name: flight-board-backend
26 strategy:
27   rollingUpdate:
28     maxSurge: 1
29     maxUnavailable: 0
30   type: RollingUpdate
31 template:
32   metadata:
33     annotations:
34       checksum/config: a681a03196a409182801f23b9c26d7126a3b427d6559d785d2c3cb42a5161c16
35     labels:
36       app.kubernetes.io/managed-by: Helm
37       app.kubernetes.io/name: flight-board-backend
38       app.kubernetes.io/part-of: flight-board
39       helm.sh/chart: flight-board-0.1.0
40 spec:
41   automountServiceAccountToken: false
42   containers:
43   - env:
44     - name: FLIGHTBOARD_VERSION
45       value: 8812e08538ae28974a856abbac3808fa93aa3f5c
46     - name: AERODATABOX_API_KEY
```

RollingUpdate with maxSurge 1 and maxUnavailable 0, so a deploy never drops below 2 ready pods. A checksum/config annotation restarts pods when the ConfigMap changes; automountServiceAccountToken is false. FLIGHTBOARD_VERSION carries the commit SHA.

```
45     value: 8812e08538ae28974a856abbac3808fa93aa3f5c
46   - name: AERODATABOX_API_KEY
47     valueFrom:
48       secretKeyRef:
49         key: api-key
50         name: flight-board-aerodatabox
51   - name: SPRING_DATA_MONGODB_URI
52     valueFrom:
53       secretKeyRef:
54         key: connectionString.standard
55         name: flight-board-mongodb-admin-flightboard-app
56   - name: JAVA_TOOL_OPTIONS
57     value: -XX:MaxRAMPercentage=75
58   envFrom:
59   - configMapRef:
60     name: flight-board-config
61   image: europe-west3-docker.pkg.dev/flight-board-prg-2610/flight-board/backend:8812e08538ae28974a856abbac3808fa93aa3f5c
62   imagePullPolicy: IfNotPresent
63   livenessProbe:
64     failureThreshold: 3
65     httpGet:
66       path: /actuator/health/liveness
67       port: management
68     scheme: HTTP
```

Secrets are referenced by name and key only: AERODATABOX_API_KEY from flight-board-aerodatabox (key api-key) and SPRING_DATA_MONGODB_URI from flight-board-mongodb-admin-flightboard-app (key connectionString.standard). All other settings come from ConfigMap flight-board-config via envFrom. Liveness probe: GET /actuator/health/liveness on the management port, failureThreshold 3.

```
67     port: management
68     scheme: HTTP
69     periodSeconds: 10
70     successThreshold: 1
71     timeoutSeconds: 1
72   name: backend
73   ports:
74   - containerPort: 8080
75     name: http
76     protocol: TCP
77   - containerPort: 8081
78     name: management
79     protocol: TCP
80   readinessProbe:
81     failureThreshold: 3
82     httpGet:
83       path: /readyz
84       port: http
85       scheme: HTTP
86     periodSeconds: 10
87     successThreshold: 1
88     timeoutSeconds: 1
89   resources:
90   limits:
```

Liveness probe runs every 10 s with a 1 s timeout. Ports: 8080 (http, API) and 8081 (management, health). Readiness probe: GET /readyz on port http (8080), every 10 s, failureThreshold 3; this is also the path the load balancer health check uses.

```
89     resources:
90       limits:
91         memory: 768Mi
92       requests:
93         cpu: 250m
94         memory: 512Mi
95     securityContext:
96       allowPrivilegeEscalation: false
97       capabilities:
98         drop:
99         - ALL
100     readOnlyRootFilesystem: true
101     startupProbe:
102       failureThreshold: 30
103       httpGet:
104         path: /actuator/health/liveness
105         port: management
106         scheme: HTTP
107       periodSeconds: 5
108       successThreshold: 1
109       timeoutSeconds: 1
110     terminationMessagePath: /dev/termination-log
111     terminationMessagePolicy: File
112     volumeMounts:
```

Requests: cpu 250m, memory 512Mi; limit: memory 768Mi. Container runs without privilege escalation, with all Linux capabilities dropped and a read-only root filesystem. Startup probe: GET /actuator/health/liveness on the management port every 5 s, failureThreshold 30 (up to 150 s for JVM start).

```
111     terminationMessagePolicy: File
112     volumeMounts:
113     - mountPath: /tmp
114       name: tmp
115     dnsPolicy: ClusterFirst
116     restartPolicy: Always
117     schedulerName: default-scheduler
118     securityContext:
119       runAsNonRoot: true
120       seccompProfile:
121         type: RuntimeDefault
122     terminationGracePeriodSeconds: 30
123     topologySpreadConstraints:
124     - labelSelector:
125         matchLabels:
126           app.kubernetes.io/name: flight-board-backend
127       maxSkew: 1
128       topologyKey: kubernetes.io/hostname
129       whenUnsatisfiable: DoNotSchedule
130     volumes:
131     - emptyDir: {}
132       name: tmp
133 status:
134   availableReplicas: 2
```

Pod runs as non-root with the RuntimeDefault seccomp profile and a writable emptyDir only at /tmp. topologySpreadConstraints: maxSkew 1 on kubernetes.io/hostname with whenUnsatisfiable: DoNotSchedule, so the two replicas must land on different nodes. No affinity or anti-affinity is set.

```
130     volumes:
131     - emptyDir: {}
132       name: tmp
133 status:
134   availableReplicas: 2
135   conditions:
136   - lastTransitionTime: '2026-10-10T01:47:16Z'
137     lastUpdateTime: '2026-10-10T01:47:16Z'
138     message: Deployment has minimum availability.
139     reason: MinimumReplicasAvailable
140     status: 'True'
141     type: Available
142   - lastTransitionTime: '2026-10-10T01:46:48Z'
143     lastUpdateTime: '2026-10-10T04:51:33Z'
144     message: ReplicaSet "flight-board-backend-74fdf7648b" has successfully progressed.
145     reason: NewReplicaSetAvailable
146     status: 'True'
147     type: Progressing
148   observedGeneration: 3
149   readyReplicas: 2
150   replicas: 2
151   terminatingReplicas: 0
152   updatedReplicas: 2
153
```

Status: availableReplicas 2, readyReplicas 2, updatedReplicas 2, observedGeneration 3; condition Available = True (MinimumReplicasAvailable) and ReplicaSet flight-board-backend-74fdf7648b has successfully progressed.

```
36     app.kubernetes.io/part-of: flight-board
37     helm.sh/chart: flight-board-0.1.0
38   spec:
39     automountServiceAccountToken: false
40     containers:
41     - image: europe-west3-docker.pkg.dev/flight-board-prg-2610/flight-board/frontend:8812e085...
42       imagePullPolicy: IfNotPresent
43       livenessProbe:
44         failureThreshold: 3
45         httpGet:
46           path: /healthz
47           port: http
48           scheme: HTTP
49         periodSeconds: 10
50         successThreshold: 1
51         timeoutSeconds: 1
52       name: frontend
53       ports:
54       - containerPort: 8080
55         name: http
56         protocol: TCP
57       readinessProbe:
58         failureThreshold: 3
59         httpGet:
```

Frontend image tag 8812e085..., imagePullPolicy IfNotPresent, container port 8080 (http). Liveness probe: GET /healthz on port http every 10 s, timeout 1 s, failureThreshold 3.

```
58     failureThreshold: 3
59     httpGet:
60       path: /healthz
61       port: http
62       scheme: HTTP
63     periodSeconds: 10
64     successThreshold: 1
65     timeoutSeconds: 1
66   resources:
67     limits:
68       memory: 128Mi
69     requests:
70       cpu: 50m
71       memory: 64Mi
72   securityContext:
73     allowPrivilegeEscalation: false
74     capabilities:
75       drop:
76       - ALL
77     readOnlyRootFilesystem: true
78   terminationMessagePath: /dev/termination-log
79   terminationMessagePolicy: File
80   volumeMounts:
81     - mountPath: /tmp
```

Readiness probe: GET /healthz on port http every 10 s. Requests cpu 50m / memory 64Mi, limit memory 128Mi. No privilege escalation, all capabilities dropped, read-only root filesystem. The frontend container has no startup probe.

```
80     volumeMounts:
81     - mountPath: /tmp
82       name: tmp
83     dnsPolicy: ClusterFirst
84     restartPolicy: Always
85     schedulerName: default-scheduler
86     securityContext:
87       runAsNonRoot: true
88       seccompProfile:
89         type: RuntimeDefault
90     terminationGracePeriodSeconds: 30
91     topologySpreadConstraints:
92     - labelSelector:
93         matchLabels:
94           app.kubernetes.io/name: flight-board-frontend
95       maxSkew: 1
96       topologyKey: kubernetes.io/hostname
97       whenUnsatisfiable: DoNotSchedule
98     volumes:
99     - emptyDir: {}
100       name: tmp
101 status:
102   availableReplicas: 2
103   conditions:
```

runAsNonRoot, RuntimeDefault seccomp, emptyDir at /tmp and the same topologySpreadConstraints as the backend: maxSkew 1 on kubernetes.io/hostname, DoNotSchedule. No affinity or anti-affinity is set.

```
98     volumes:
99     - emptyDir: {}
100       name: tmp
101 status:
102   availableReplicas: 2
103   conditions:
104   - lastTransitionTime: '2026-10-10T01:46:56Z'
105     lastUpdateTime: '2026-10-10T01:46:56Z'
106     message: Deployment has minimum availability.
107     reason: MinimumReplicasAvailable
108     status: 'True'
109     type: Available
110   - lastTransitionTime: '2026-10-10T01:46:48Z'
111     lastUpdateTime: '2026-10-10T04:51:03Z'
112     message: ReplicaSet "flight-board-frontend-8566b9795" has successfully progressed.
113     reason: NewReplicaSetAvailable
114     status: 'True'
115     type: Progressing
116   observedGeneration: 3
117   readyReplicas: 2
118   replicas: 2
119   terminatingReplicas: 0
120   updatedReplicas: 2
121
```

Status: availableReplicas 2, readyReplicas 2, updatedReplicas 2, observedGeneration 3; ReplicaSet flight-board-frontend-8566b9795 has successfully progressed.

▼ policy	Toggle node	API Group				
▼ PodDisruptionBudget		Kind				
flight-board-backend	● Unknown	PodDisruptionBudget	flight-board	flight-board	europa-west3-a	
flight-board-frontend	● Unknown	PodDisruptionBudget	flight-board	flight-board	europa-west3-a	
flight-board-mongodb	● Unknown	PodDisruptionBudget	flight-board	flight-board	europa-west3-a	

Three PodDisruptionBudgets exist in namespace flight-board: flight-board-backend, flight-board-frontend and flight-board-mongodb. The Object Browser shows their status column as Unknown; the real status is in the YAML on the next pages.

```
13   name: flight-board-backend
14   namespace: flight-board
15   resourceVersion: '1791607895250127000'
16   uid: daa7773f-fee4-4c93-8cab-663828383f63
17 spec:
18   minAvailable: 1
19   selector:
20     matchLabels:
21       app.kubernetes.io/name: flight-board-backend
22 status:
23   conditions:
24     - lastTransitionTime: '2026-10-10T01:47:15Z'
25       message: ''
26       observedGeneration: 1
27       reason: SufficientPods
28       status: 'True'
29       type: DisruptionAllowed
30   currentHealthy: 2
31   desiredHealthy: 1
32   disruptionsAllowed: 1
33   expectedPods: 2
34   observedGeneration: 1
35
```

```
13   name: flight-board-Frontend
14   namespace: flight-board
15   resourceVersion: '1791607864394111002'
16   uid: fc8012d8-0cee-4b2c-9f58-2da017e69c94
17 spec:
18   minAvailable: 1
19   selector:
20     matchLabels:
21       app.kubernetes.io/name: flight-board-frontend
22 status:
23   conditions:
24     - lastTransitionTime: '2026-10-10T01:46:56Z'
25       message: ''
26       observedGeneration: 1
27       reason: SufficientPods
28       status: 'True'
29       type: DisruptionAllowed
30   currentHealthy: 2
31   desiredHealthy: 1
32   disruptionsAllowed: 1
33   expectedPods: 2
34   observedGeneration: 1
35
```

Backend (left) and frontend (right): minAvailable 1, currentHealthy 2, desiredHealthy 1, disruptionsAllowed 1, expectedPods 2. A node drain can evict only one pod of each at a time.

```
11   name: flight-board-mongodb
12   namespace: flight-board
13   resourceVersion: '1791595124045007024'
14   uid: e1bcfb41-86ff-4244-842e-e329c36bb797
15 spec:
16   minAvailable: 2
17   selector:
18     matchLabels:
19       app.kubernetes.io/name: flight-board-mongodb
20 status:
21   conditions:
22   - lastTransitionTime: '2026-10-10T01:18:44Z'
23     message: ''
24     observedGeneration: 1
25     reason: SufficientPods
26     status: 'True'
27     type: DisruptionAllowed
28   currentHealthy: 3
29   desiredHealthy: 2
30   disruptionsAllowed: 1
31   expectedPods: 3
32   observedGeneration: 1
33
```

MongoDB: minAvailable 2 of 3 members, currentHealthy 3, disruptionsAllowed 1, so the replica set keeps a majority during node maintenance.

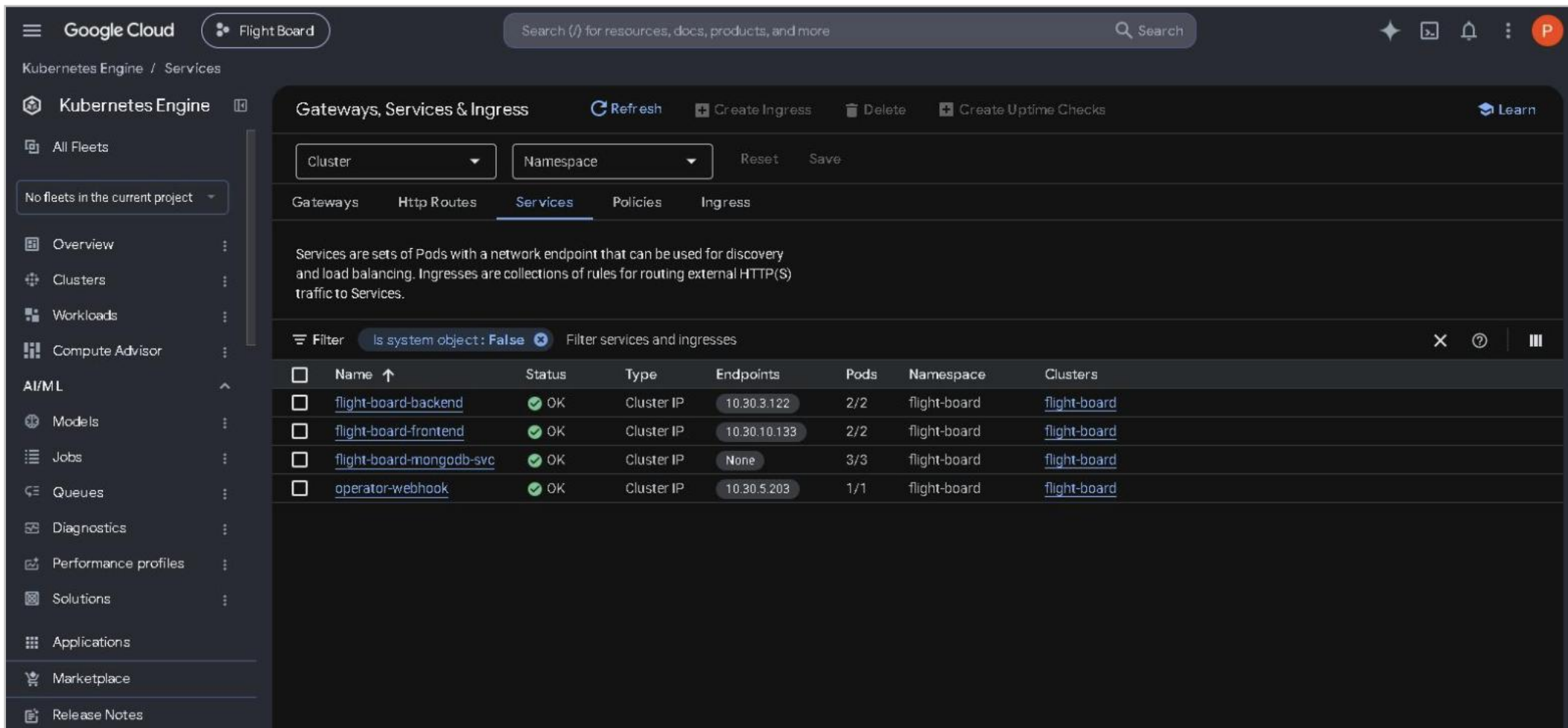
Data	
FLIGHTBOARD_ADMIN_ADDRESS	127.0.0.1
FLIGHTBOARD_ADMIN_PORT	8082
FLIGHTBOARD_AERODATABOX_BASE_URL	https://aerodatabox.p.rapidapi.com
FLIGHTBOARD_AERODATABOX_HOST	aerodatabox.p.rapidapi.com
FLIGHTBOARD_BOARD_MAX_AGE	PT12H
FLIGHTBOARD_BOARD_MAX_FLIGHTS	36
FLIGHTBOARD_BOARD_STALE_AFTER	PT75M
FLIGHTBOARD_BOARD_TERMINAL	2
FLIGHTBOARD_BOARD_TIMEZONE	Europe/Prague
FLIGHTBOARD_FETCH_CHECK_INTERVAL	PT1M
FLIGHTBOARD_FETCH_INTERVAL	PT30M
FLIGHTBOARD_FETCH_LOCK_DURATION	PT5M
FLIGHTBOARD_FETCH_RUN_TIMEOUT	PT2M
FLIGHTBOARD_MONGODB_DATABASE	flightboard
FLIGHTBOARD_SOURCE	aerodatabox
FLIGHTBOARD_SOURCE_AIRPORT	PRG
FLIGHTBOARD_SOURCE_DURATION_MINUTES	720
FLIGHTBOARD_SOURCE_OFFSET_MINUTES	-60
MANAGEMENT_SERVER_PORT	8081
SERVER_PORT	8080

Fetch interval PT30M, stale after PT75M, maximum board age PT12H, airport PRG, terminal 2, time zone Europe/Prague, at most 36 flights, fetch check every PT1M, lock PT5M, run timeout PT2M, source window 720 minutes starting 60 minutes back. Ports 8080 (API) and 8081 (management); admin port 8082 listens only on 127.0.0.1.

☐	Name ↑	Type	Namespace	Cluster
☐	flight-board-aerodatabox	Secret	flight-board	flight-board
☐	flight-board-config	Config Map	flight-board	flight-board
☐	flight-board-mongodb-admin-flightboard-app	Secret	flight-board	flight-board
☐	flight-board-mongodb-agent-password	Secret	flight-board	flight-board
☐	flight-board-mongodb-app-password	Secret	flight-board	flight-board
☐	flight-board-mongodb-app-scram-credentials	Secret	flight-board	flight-board
☐	flight-board-mongodb-config	Secret	flight-board	flight-board
☐	flight-board-mongodb-keyfile	Secret	flight-board	flight-board
☐	flight-board-tls	Secret	flight-board	flight-board

☐	flight-board-mongodb-agent-password	Secret	flight-board	flight-board
☐	flight-board-mongodb-app-password	Secret	flight-board	flight-board
☐	flight-board-mongodb-app-scram-credentials	Secret	flight-board	flight-board
☐	flight-board-mongodb-config	Secret	flight-board	flight-board
☐	flight-board-mongodb-keyfile	Secret	flight-board	flight-board
☐	flight-board-tls	Secret	flight-board	flight-board
☐	kube-root-ca.crt	Config Map	kube-public	flight-board
☐	kube-root-ca.crt	Config Map	kube-node-lease	flight-board
☐	kube-root-ca.crt	Config Map	default	flight-board
☐	kube-root-ca.crt	Config Map	flight-board	flight-board
☐	mongodb-enterprise-operator-telemetry	Config Map	flight-board	flight-board
☐	sh.helm.release.v1.flight-board.v1	Secret	flight-board	flight-board
☐	sh.helm.release.v1.flight-board.v2	Secret	flight-board	flight-board
☐	sh.helm.release.v1.flight-board.v3	Secret	flight-board	flight-board
☐	sh.helm.release.v1.mongodb-kubernetes.v1	Secret	flight-board	flight-board

Secret names and types only; no values were opened. App secrets: flight-board-aerodatabox, flight-board-mongodb-app-password, flight-board-tls. Operator-generated: flight-board-mongodb-admin-flightboard-app, -agent-password, -app-scram-credentials, -config, -keyfile. Helm keeps release history in sh.helm.release.v1.* Secrets. Key names visible in references: api-key and connectionString.standard (backend YAML), password (MongoDBCommunity resource).



The screenshot shows the Google Cloud console interface for Kubernetes Engine. The left sidebar contains navigation links for 'Kubernetes Engine', 'All Fleets', 'Overview', 'Clusters', 'Workloads', 'Compute Advisor', 'AI/ML', 'Models', 'Jobs', 'Queues', 'Diagnostics', 'Performance profiles', 'Solutions', 'Applications', 'Marketplace', and 'Release Notes'. The main content area is titled 'Gateways, Services & Ingress' and includes a 'Refresh' button and links to 'Create Ingress', 'Delete', and 'Create Uptime Checks'. Below this, there are filters for 'Cluster' and 'Namespace', and a 'Reset' button. The 'Services' tab is selected, showing a table of services. The table has columns for Name, Status, Type, Endpoints, Pods, Namespace, and Clusters. The services listed are 'flight-board-backend', 'flight-board-frontend', 'flight-board-mongodb-svc', and 'operator-webhook', all with status 'OK' and type 'Cluster IP'. The 'flight-board-mongodb-svc' service has 'None' as its endpoint, indicating it is a headless service.

Name	Status	Type	Endpoints	Pods	Namespace	Clusters
flight-board-backend	OK	Cluster IP	10.30.3.122	2/2	flight-board	flight-board
flight-board-frontend	OK	Cluster IP	10.30.10.133	2/2	flight-board	flight-board
flight-board-mongodb-svc	OK	Cluster IP	None	3/3	flight-board	flight-board
operator-webhook	OK	Cluster IP	10.30.5.203	1/1	flight-board	flight-board

ClusterIP services select the backend and frontend pods; the Ingress refers to them and the NEG controller registers their pod IPs in the load balancer. MongoDB has a stable headless service.

The screenshot shows the Google Cloud console interface for the 'Flight Board' project. The left sidebar contains navigation links for Kubernetes Engine, All Fleets, Overview, Clusters, Workloads, Compute Advisor, AI/ML, Models, Jobs, Queues, Diagnostics, Performance profiles, Solutions, Applications, Marketplace, and Release Notes. The main content area is titled 'Gateways, Services & Ingress' and includes a 'Refresh' button and a 'Delete' button. Below this, there are filters for 'Cluster' and 'Namespace', and buttons for 'Reset' and 'Save'. The 'Ingress' tab is selected, showing a message: 'An Ingress is a collection of rules that allow inbound connections to reach the cluster services.' Below this message is a warning: 'The Ingress-NGINX project has been retired and is no longer supported, consider upgrading to GKE Gateway [Learn more](#)'. A table below the warning lists the Ingress configuration:

Name	Status	Type	Frontends	Services	Namespace	Clusters
flight-board	OK	External HTTP(S) LB	flights.peterhudcovic.tech/api	flight-board-backend , flight-...	flight-board	flight-...

The Ingress is configuration, not a network hop: from it GKE builds the external load balancer for `flights.peterhudcovic.tech`, with `/api` to the backend and everything else to the frontend.

Google Cloud Flight Board Search (/) for resources, docs, products, and more Search

Network Services

- Load balancing
- Cloud DNS
- Cloud CDN
- Cloud NAT
- Cloud Service Mesh (Tra...
- Service Directory
- Cloud Domains
- Private Service Connect
- Composite Health
- SSL policies
- Service Extensions
- Global Front End Preview
- Marketplace
- Release Notes

Load balancing + Create load balancer Refresh Delete Learn

Load balancers Backends Frontends Service LB policies

ⓘ Faster web performance and improved web protection with Cloud CDN and Cloud Armor. [Learn more](#) Dismiss

Filter Enter property name or value ? |||

☐	Name	Load balancer type	Access type	Protocols	Region	Backends	Actions
☐	k8s2-um-1f1lqt48-flight-board-flight-board-kvq43txt	Application (Classic)	External	HTTPS		✔ 3 backend services (0 instance groups, 3 network endpoint groups)	⋮

To view or delete load balancing resources like forwarding rules and target proxies, go to the [load balancing components view](#).

Google Cloud created the external Application Load Balancer from the Ingress; it sends HTTPS traffic to three network endpoint groups (container-native load balancing).

k8s2-um-1f1iq48-flight-board-flight-board-kvq43txt

Classic Application Load Balancer

 Faster web performance and improved web protection with Cloud CDN and Cloud Armor. [Learn more](#) Dismiss

[Details](#) [Monitoring](#) [Cache invalidation](#) [Migration](#) [New](#)

Frontend

Protocol 	IP:Port	Certificate	Certificate Map	SSL Policy	Network Tier 
HTTPS	35.190.55.104:443	k8s2-cr-1f1iq48-m08h2bdy7feq75sm-4e76e534ee7d219a		GCP default	Premium

k8s2-um-1f1iq48-flight-board-flight-board-kvq43txt is a Classic Application Load Balancer. Its frontend is HTTPS on 35.190.55.104:443 with a configured TLS certificate, GCP default SSL policy and Premium network tier. The certificate is shown on the next page.

☐	Name ↑	Domain	Expires	Type	Scope	Status	In use by
☐	k8s2-cr-1f1iq48-m08h2bdy7feq75sm-4e76e534ee7d219a	flights.peterhudcovic.tech	Oct 6, 2041, 3:25:00 AM	Self managed	Global		k8s2-ts-1f1iq48-flight-board- 

k8s2-cr-1f1iq48-m08h2bdy7feq75sm-4e76e534ee7d219a

DNS Hostnames	flights.peterhudcovic.tech
Expires	✓ Oct 6, 2041, 3:25:00 AM
Serial number	5B:EB:66:7C:7C:2B:24:BB:EE:18:FA:B0:AB:D4:6E:06:39:9A:C4:B2
Certificate Issuer	
Tags	— 

In use by
[k8s2-ts-1f1iq48-flight-board-flight-board-kvq43txt](#)

Certificate chain
 CloudFlare Origin Certificate - Oct 6, 2041, 3:25:00 AM

[Equivalent REST](#)

The load balancer certificate is Self managed, not Google-managed: its chain is a CloudFlare Origin Certificate for flights.peterhudcovic.tech, valid until 6 Oct 2041. GKE uploads it from Secret flight-board-tls. It is trusted by Cloudflare, which is why the DNS record must stay proxied.

Host and path rules

Hosts ↑	Paths	Backend
All unmatched (default)	All unmatched (default)	k8s1-d68af731-kube-system-default-http-backend-80-24a9950d
flights.peterhudcovic.tech	/*	k8s1-d68af731-flight-board-flight-board-frontend-80-e16bcb17
flights.peterhudcovic.tech	/api/*	k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b
flights.peterhudcovic.tech	/api	k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b
flights.peterhudcovic.tech	All unmatched (default)	k8s1-d68af731-kube-system-default-http-backend-80-24a9950d

For host `flights.peterhudcovic.tech`: `/*` goes to the frontend backend service, `/api` and `/api/*` go to the backend backend service. Every unmatched host or path goes to kube-system default-http-backend.

Backend

Backend services

1. k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b

Endpoint protocol	HTTP
Timeout	30 seconds
Health check	k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b
Cloud CDN	Disabled
Logging	Enabled (sample rate: 1) All optional fields excluded
Backend security policy	None

[Advanced configurations](#)

Backends

Name 	Type	Scope	Healthy	Autoscaling	Balancing mode	Capacity	Preference level 
k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b	Zonal network endpoint group	europe-west3-a	 2 of 2 View Details	No configuration	Max RPS: 1 (per endpoint)	100%	None

2. k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b

k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b: HTTP to the pods, 30 s timeout, Cloud CDN disabled, request logging enabled (sample rate 1). Its backend is a zonal network endpoint group in europe-west3-a with 2 of 2 endpoints healthy.

2. k8s1-d68af731-flight-board-flight-board-frontend-80-e16bcb17

Endpoint protocol	HTTP
Timeout	30 seconds
Health check	k8s1-d68af731-flight-board-flight-board-frontend-80-e16bcb17
Cloud CDN	Disabled
Logging	Enabled (sample rate: 1) All optional fields excluded
Backend security policy	None

▼ Advanced configurations

Backends

Name ↑	Type	Scope	Healthy	Autoscaling	Balancing mode	Capacity	Preference level ?
k8s1-d68af731-flight-board-flight-board-frontend-80-e16bcb17	Zonal network endpoint group	europe-west3-a	✔ 2 of 2 View Details	No configuration	Max RPS: 1 (per endpoint)	100%	None

k8s1-d68af731-flight-board-flight-board-frontend-80-e16bcb17: same settings, its zonal network endpoint group reports 2 of 2 healthy. Below it starts the third backend service for the kube-system default backend.

Health checks
Create health check
Refresh
Delete
Learn

Health checks determine if applications on your VMs respond to requests. They're used for load balancing and with autohealing in managed instance groups. [Learn more](#)

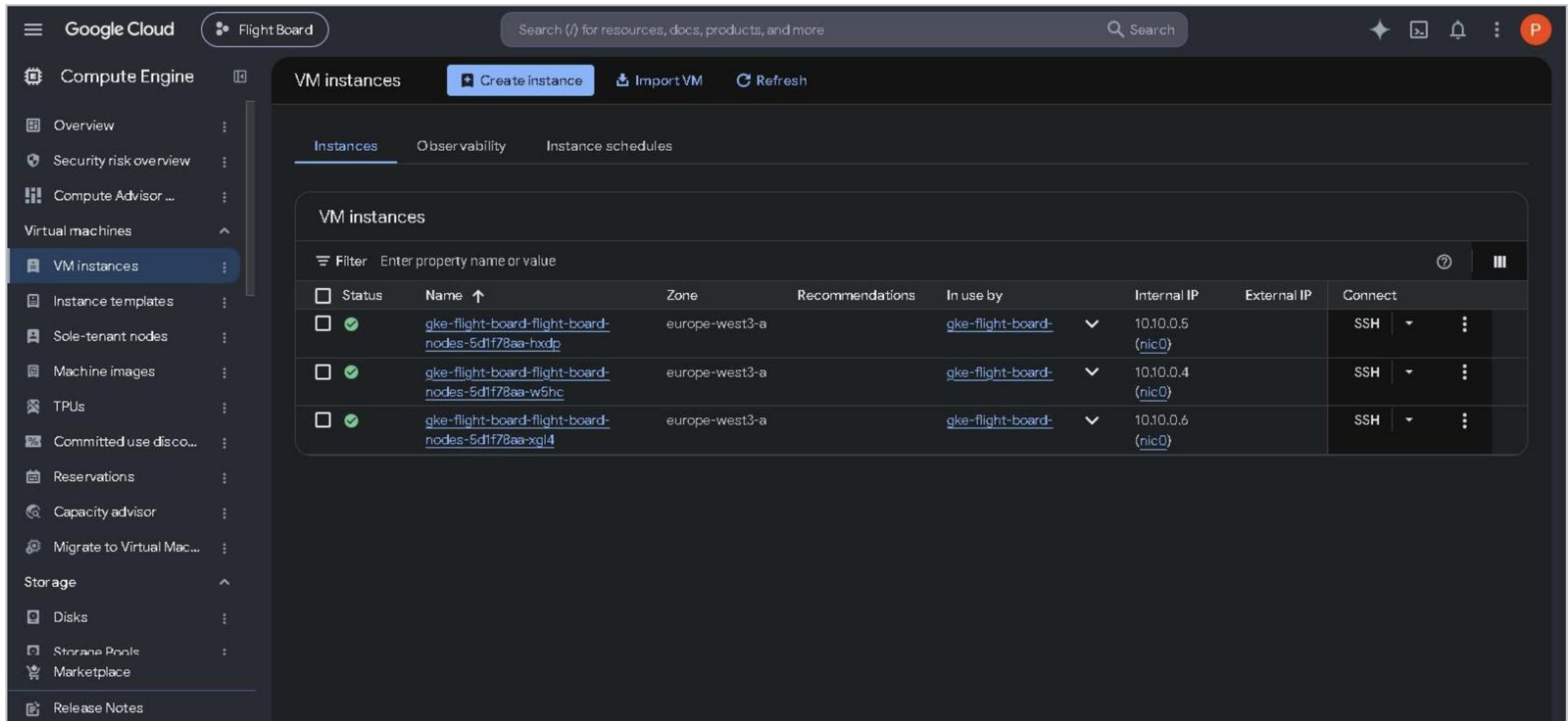
Filter
Enter property name or value

☐	Name ↑	Scope	Region	Host	Path	Protocol	Port	In use by
☐	k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b	Global			/readyz	HTTP	8080	k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b
☐	k8s1-d68af731-flight-board-flight-board-frontend-80-e16bcb17	Global			/healthz	HTTP	8080	k8s1-d68af731-flight-board-flight-board-frontend-80-e16bcb17
☐	k8s1-d68af731-kube-system-default-http-backend-80-24a9950d	Global			/healthz	HTTP	80	k8s1-d68af731-kube-system-default-http-backend-80-24a9950d

Health checks used by the load balancer: backend GET /readyz on port 8080, frontend GET /healthz on port 8080, kube-system default backend GET /healthz on port 80; all HTTP.

Network endpoint group + Create network endpoint group Refresh Delete									
Filter Enter property name or value ? ⋮									
☐	Name ↑	Type	Network endpoints	Scope	Subnet	VPC network	In use by		
☐	k8s1-d68af731-flight-board-flight-board-backend-80-efd29d1b	Zonal NEG	2	Zonal (europe-west3-a)	flight-board-subnet	flight-board-vpc	k8s1-d68af731-flight-board-	▼	🗑
☐	k8s1-d68af731-flight-board-flight-board-frontend-80-e16bcb17	Zonal NEG	2	Zonal (europe-west3-a)	flight-board-subnet	flight-board-vpc	k8s1-d68af731-flight-board-	▼	🗑
☐	k8s1-d68af731-kube-system-default-http-backend-80-24a9950d	Zonal NEG	1	Zonal (europe-west3-a)	flight-board-subnet	flight-board-vpc	k8s1-d68af731-kube-system-	▼	🗑

Three zonal NEGs in flight-board-vpc / flight-board-subnet: backend with 2 endpoints, frontend with 2 endpoints and the default backend with 1. Each endpoint is a pod IP, so traffic goes straight to pods.



The screenshot displays the Google Cloud console interface for the 'Flight Board' project. The left sidebar contains a navigation menu with the following items: Compute Engine, Overview, Security risk overview, Compute Advisor..., Virtual machines, VM instances (selected), Instance templates, Sole-tenant nodes, Machine images, TPU's, Committed use disco..., Reservations, Capacity advisor, Migrate to Virtual Mac..., Storage, Disks, Storage Pools, Marketplace, and Release Notes. The main panel is titled 'VM instances' and includes buttons for 'Create instance', 'Import VM', and 'Refresh'. Below the title bar, there are tabs for 'Instances', 'Observability', and 'Instance schedules'. The 'Instances' tab is active, showing a table of VM instances. The table has columns for Status, Name, Zone, Recommendations, In use by, Internal IP, External IP, and Connect. There are three instances listed, all with a status of 'Running' (indicated by a green checkmark) and located in the 'europe-west3-a' zone. All three instances have internal IP addresses (10.10.0.5, 10.10.0.4, and 10.10.0.6) and no external IP addresses. The 'Connect' column shows an 'SSH' button and a dropdown menu for each instance.

Status	Name	Zone	Recommendations	In use by	Internal IP	External IP	Connect
Running	gke-flight-board-flight-board-nodes-5d1f78aa-hxdp	europe-west3-a		gke-flight-board-	10.10.0.5 (nic0)		SSH
Running	gke-flight-board-flight-board-nodes-5d1f78aa-w5hc	europe-west3-a		gke-flight-board-	10.10.0.4 (nic0)		SSH
Running	gke-flight-board-flight-board-nodes-5d1f78aa-xgl4	europe-west3-a		gke-flight-board-	10.10.0.6 (nic0)		SSH

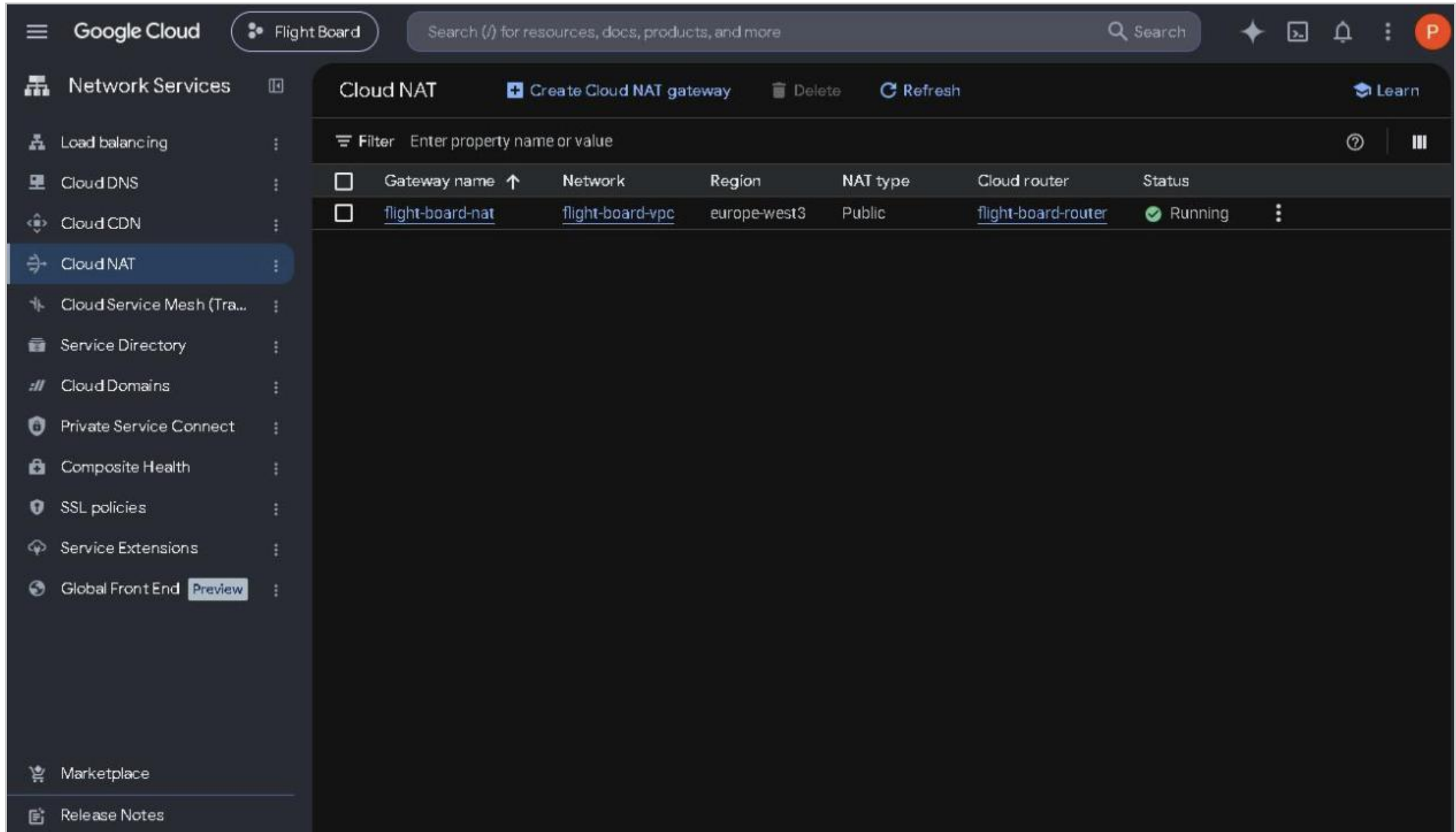
The three node VMs have only internal IPs (10.10.0.x) and no external IP, so nothing on the nodes is directly reachable from the internet.

The screenshot shows the Google Cloud VPC networks page. The left sidebar contains a navigation menu with options like VPC Network, VPC networks, IP addresses, Internal ranges, Bring your own IP, Firewall, Routes, VPC connectivity, Shared VPC, Serverless VPC access, Packet mirroring, and VPC Flow Logs. The main content area is titled 'VPC networks' and has tabs for 'Networks in current project' and 'Subnets in current project'. The 'Subnets in current project' tab is active, showing a table of subnets. A dropdown menu for 'VPC networks' is open, showing 'flight-board-vpc'. The table has columns for Name, Region, VPC network, IP stack type, Primary IPv4 range, Secondary IPv4 ranges, IPv6 ranges, Gateways, and Flow log config. The first row shows a subnet named 'flight-board-subnet' in the 'europe-west3' region, associated with the 'flight-board-vpc' network, with a primary IPv4 range of '10.10.0.0/24' and a secondary range of '10.20.0.0/16'. Below the table, there is a section for 'Reserved proxy-only subnets for load balancing' with columns for Name, Region, VPC network, IP address ranges, Gateway, Role, and Purpose. The table is currently empty, showing 'No rows to display'.

Name	Region	VPC network	IP stack type	Primary IPv4 range	Secondary IPv4 ranges	IPv6 ranges	Gateways	Flow log config
flight-board-subnet	europe-west3	flight-board-vpc	IPv4	10.10.0.0/24	10.20.0.0/16		10.10.0.1	

Name	Region	VPC network	IP address ranges	Gateway	Role	Purpose
No rows to display						

A custom VPC with one subnet for nodes (10.10.0.0/24) and secondary ranges for pods and services, so pods get VPC-native IPs.



The screenshot shows the Google Cloud console interface. On the left is a sidebar with 'Network Services' expanded, listing various services like Load balancing, Cloud DNS, Cloud CDN, Cloud NAT (selected), Cloud Service Mesh, Service Directory, Cloud Domains, Private Service Connect, Composite Health, SSL policies, Service Extensions, and Global Front End. The main panel displays the 'Cloud NAT' configuration for a resource named 'flight-board-nat'. At the top of the main panel, there are buttons for 'Create Cloud NAT gateway', 'Delete', 'Refresh', and a 'Learn' link. Below these is a filter bar. The configuration is shown in a table with the following columns: Gateway name, Network, Region, NAT type, Cloud router, and Status. The row for 'flight-board-nat' shows it is connected to 'flight-board-vpc' in the 'europe-west3' region, with a 'Public' NAT type, using the 'flight-board-router' cloud router, and is currently in a 'Running' status.

Gateway name	Network	Region	NAT type	Cloud router	Status
flight-board-nat	flight-board-vpc	europe-west3	Public	flight-board-router	Running

Cloud NAT on the flight-board router gives the private nodes outbound internet access to pull images and call the AeroDataBox API.

Google Cloud Flight Board

Search (/) for resources, docs, products, and more

Network Security

Firewall policies

Create firewall policy Create firewall rule

Filter Enter property name or value

Name	Type	Targets	Filters	Protocols / ports	Action	Priority	Network
default-allow-icmp	Ingress	Apply to all	IP ranges:	icmp	Allow	65534	default
default-allow-internal	Ingress	Apply to all	IP ranges:	tcp:0-65535 udp:0-65535 icmp	Allow	65534	default
default-allow-rdp	Ingress	Apply to all	IP ranges:	tcp:3389	Allow	65534	default
default-allow-ssh	Ingress	Apply to all	IP ranges:	tcp:22	Allow	65534	default
flight-board-allow-lb-hc	Ingress	flight-board-node	IP ranges:	tcp:8080	Allow	1000	flight-board-vpc
gke-flight-board-18f0575d-all	Ingress	gke-flight-board-18f0575d-node	IP ranges:	tcp udp sctp icmp;esp;ah	Allow	1000	flight-board-vpc
gke-flight-board-18f0575d-vms	Ingress	gke-flight-board-18f0575d-node	IP ranges:	tcp:1-65535 udp:1-65535 icmp	Allow	1000	flight-board-vpc
k8s-fw-l7-d68af73174ca2be9	Ingress	gke-flight-board-18f0575d-node	IP ranges:	tcp:8080	Allow	1000	flight-board-vpc

Network firewall policies

Firewall policies let you group several firewall rules so that you can update them all at once, effectively controlled by Identity and Access Management (IAM) roles.

[Learn more](#)

The flight-board-allow-lb-hc rule opens only tcp:8080 from Google health-check ranges to the nodes; the default network rules belong to the unused default VPC.

Google Cloud Flight Board

Kubernetes Engine / Persistent volume claims

Storage Refresh Delete Learn

Cluster Namespace Reset Save

Persistent volume claims Storage classes

Persistent volume claims are requests for storage of specific size and access mode. [Learn more](#)

Filter Filter persistent volume claims

☐	Name ↑	Phase	Volume	Storage class	Namespace	Cluster
☐	data-volume-flight-board-mongodb-0	Bound	pvc-533af496-5028-4bd9-8420-ad0ae470201e	standard-rwo	flight-board	flight-board
☐	data-volume-flight-board-mongodb-1	Bound	pvc-18807e6f-473b-433a-815a-39cf098b8d43	standard-rwo	flight-board	flight-board
☐	data-volume-flight-board-mongodb-2	Bound	pvc-f09f6aab-0a12-426e-84e3-bdfb2279e33f	standard-rwo	flight-board	flight-board
☐	logs-volume-flight-board-mongodb-0	Bound	pvc-3e087eec-2d91-4e50-9741-520ec4b8a414	standard-rwo	flight-board	flight-board
☐	logs-volume-flight-board-mongodb-1	Bound	pvc-4958eeb0-e952-41ec-ba9f-3cee0e99f21a	standard-rwo	flight-board	flight-board
☐	logs-volume-flight-board-mongodb-2	Bound	pvc-513d2186-e53d-406e-816d-36e0b01e3a55	standard-rwo	flight-board	flight-board

Six persistent volume claims, a data and a logs disk for each MongoDB member, all Bound on standard-rwo (pd-balanced) disks.

```
34     affinity:
35       podAntiAffinity:
36         requiredDuringSchedulingIgnoredDuringExecution:
37           - labelSelector:
38               matchLabels:
39                 app.kubernetes.io/name: flight-board-mongodb
40             topologyKey: kubernetes.io/hostname
41     containers:
42     - name: mongod
43       resources:
44         limits:
45           memory: 1Gi
46         requests:
47           cpu: 250m
48           memory: 512Mi
49     volumeClaimTemplates:
50     - metadata:
51         labels:
52           app.kubernetes.io/part-of: flight-board
53         name: data-volume
54       spec:
55         accessModes:
56         - ReadWriteOnce
57       resources:
58         requests:
59           storage: 5Gi
60       storageClassName: standard-rwo
```

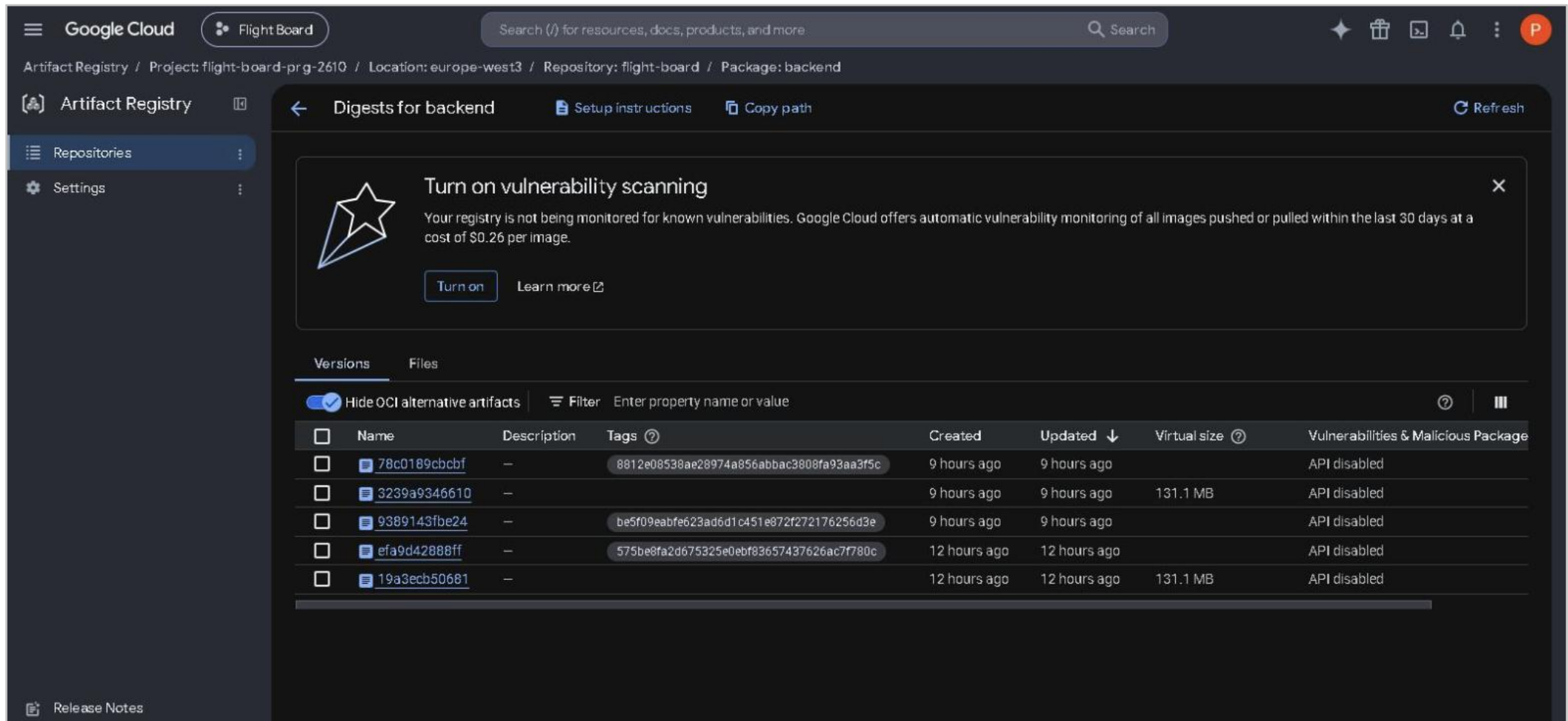
MongoDBCommunity flight-board-mongodb: required podAntiAffinity on kubernetes.io/hostname, so the three members must run on three different nodes. Container mongod requests cpu 250m / memory 512Mi, limit memory 1Gi. Data volume 5Gi on standard-rwo.

```
72   type: ReplicaSet
73   users:
74   - connectionStringSecretName: flight-board-mongodb-admin-flightboard-app
75     db: admin
76     name: flightboard-app
77     passwordSecretRef:
78       key: password
79       name: flight-board-mongodb-app-password
80     roles:
81     - db: flightboard
82       name: readWrite
83     scramCredentialsSecretName: flight-board-mongodb-app
84   version: 8.0.32
85 status:
86   currentMongoDBMembers: 3
87   currentStatefulSetReplicas: 3
88   mongoUri: mongodb://flight-board-mongodb-0.flight-board-mongodb-svc.flight-board.svc.cluster.local:27017,flight-board-mongodb-1.flight-board-mongodb-svc.flight-board.svc.cluster.local:27017,flight-board-mongodb-2.flight-board-mongodb-svc.flight-board.svc.cluster.local:27017
89   phase: Running
90   version: 8.0.32
91
```

Type ReplicaSet, MongoDB 8.0.32. App user flightboard-app has readWrite on database flightboard; its password comes from Secret flight-board-mongodb-app-password (key password), the operator writes the connection string to flight-board-mongodb-admin-flightboard-app. Status: phase Running, 3 current members, 3 StatefulSet replicas.

```
PS C:\Users\peter> kubectl get pods -n flight-board -o wide -l app=flight-board-mongodb-svc
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
flight-board-mongodb-0             2/2     Running   0          14h   10.20.2.4       gke-flight-board-flight-board-nodes-5d1f78aa-xgl4   <none>           <none>
flight-board-mongodb-1             2/2     Running   0          14h   10.20.0.4       gke-flight-board-flight-board-nodes-5d1f78aa-w5hc   <none>           <none>
flight-board-mongodb-2             2/2     Running   0          13h   10.20.1.10      gke-flight-board-flight-board-nodes-5d1f78aa-hxdp   <none>           <none>
PS C:\Users\peter> foreach ($i in 0..2) { kubectl exec -n flight-board flight-board-mongodb-$i -c mongod -- mongosh --quiet --eval "const h=db.hello(); print(h.me + ' primary=' + h.isWritablePrimary + ' secondary=' + h.secondary + ' set=' + h.setName)" }
Warning: Could not access file: EACCES: permission denied, mkdir '/data/db/.mongodb'
flight-board-mongodb-0.flight-board-mongodb-svc.flight-board.svc.cluster.local:27017 primary=false secondary=true set=flight-board-mongodb
Warning: Could not access file: EACCES: permission denied, mkdir '/data/db/.mongodb'
flight-board-mongodb-1.flight-board-mongodb-svc.flight-board.svc.cluster.local:27017 primary=true secondary=false set=flight-board-mongodb
Warning: Could not access file: EACCES: permission denied, mkdir '/data/db/.mongodb'
flight-board-mongodb-2.flight-board-mongodb-svc.flight-board.svc.cluster.local:27017 primary=false secondary=true set=flight-board-mongodb
PS C:\Users\peter>
```

Read-only kubectl output (pasted as text and rendered here). The three members run on three different nodes, each with 2/2 containers (mongod and the operator agent), 0 restarts. db.hello() on each member: flight-board-mongodb-1 (node ...-w5hc) is primary, members 0 and 2 are secondary, replica set flight-board-mongodb. The EACCES warning only means mongosh could not create its history folder.



The screenshot shows the Google Cloud Artifact Registry interface. The breadcrumb navigation indicates the path: Artifact Registry / Project: flight-board-prg-2610 / Location: europe-west3 / Repository: flight-board / Package: backend. The left sidebar contains links to Repositories and Settings. The main content area is titled 'Digests for backend' and includes links for Setup instructions and Copy path. A notification banner at the top prompts the user to 'Turn on vulnerability scanning', stating that the registry is not being monitored for known vulnerabilities and that Google Cloud offers automatic vulnerability monitoring of all images pushed or pulled within the last 30 days at a cost of \$0.26 per image. Below the notification, there are tabs for Versions and Files. The Versions tab is active, showing a table of image digests. The table has columns for Name, Description, Tags, Created, Updated, Virtual size, and Vulnerabilities & Malicious Package. The table lists five digests, all of which are API disabled.

Name	Description	Tags	Created	Updated	Virtual size	Vulnerabilities & Malicious Package
78c0189cbcbf	—	8812e08538ae28974a856abbac3808fa93aa3f5c	9 hours ago	9 hours ago		API disabled
3239a9346610	—		9 hours ago	9 hours ago	131.1 MB	API disabled
9389143fbc24	—	be5f09eabfe623ad6d1c451e872f272176256d3e	9 hours ago	9 hours ago		API disabled
efa9d42888ff	—	575be8fa2d675325e0ebf83657437626ac7f780c	12 hours ago	12 hours ago		API disabled
19a3ecb50681	—		12 hours ago	12 hours ago	131.1 MB	API disabled

GitHub Actions pushes backend and frontend images to Artifact Registry tagged with the full git commit SHA, so every running version is traceable to a commit.

The screenshot shows the GitHub Actions interface for the repository 'flight-board' by user 'PeterHudcovic'. The left sidebar contains navigation links: Code, Issues, Pull requests, Agents, Actions (selected), Projects, Wiki, Security and quality, Insights, and Settings. Below these are links to CI, Deploy, Management, Caches, Attestations, Runners, Usage metrics, and Performance metrics. The main area is titled 'All workflows' and shows 'Showing runs from all workflows'. A search bar 'Filter workflow runs' is present. Below this is a table of '9 workflow runs' with columns for Workflow, Event, Status, Branch, and Actor. The runs are as follows:

Workflow	Event	Status	Branch	Actor
Merge pull request #23 from PeterHudcovic/codex/22-board-polish	Deploy #3: Commit 8812e08 pushed by PeterHudcovic	Success	main	PeterHudcovic
Merge pull request #25 from PeterHudcovic/claude/24-destination-cont...	Deploy #2: Commit be5f09e pushed by PeterHudcovic	Success	main	PeterHudcovic
Polish departure board for live demo	CI #6: Pull request #23 synchronize by PeterHudcovic	Success	codex/22-board-polish	PeterHudcovic
Contract: destination override map (section 8)	CI #5: Pull request #25 opened by PeterHudcovic	Success	claude/24-destination-con...	PeterHudcovic
Polish departure board for live demo	CI #4: Pull request #23 opened by PeterHudcovic	Success	codex/22-board-polish	PeterHudcovic
Merge pull request #20 from PeterHudcovic/claude/19-actions	Deploy #1: Commit 575be8f pushed by PeterHudcovic	Success	main	PeterHudcovic

CI runs on every pull request and Deploy runs on every merge to main; all runs are green, the latest deploy took about 3 minutes.

Deploy
deploy.yml

Filter workflow runs

3 workflow runs

Event Status Branch Actor

This workflow has a workflow_dispatch event trigger. Run workflow

✓ Merge pull request #23 from PeterHudcovic/codex/22-board-polish Deploy #3: Commit 8812e08 pushed by PeterHudcovic	main	Today at 6:48 AM 3m 5s	...
✓ Merge pull request #25 from PeterHudcovic/claude/24-destination-contr... Deploy #2: Commit be5f09e pushed by PeterHudcovic	main	Today at 6:35 AM 2m 21s	...
✓ Merge pull request #20 from PeterHudcovic/claude/19-actions Deploy #1: Commit 575be8f pushed by PeterHudcovic	main	Today at 3:44 AM 10m 21s	...

All three Deploy runs succeeded. Each was started by a push to main after a merged pull request; the newest, run #3 for commit 8812e08 (PR #23), took 3 min 5 s.

← Deploy

✓ Merge pull request #23 from PeterHudcovic/codex/22-board-polish #3

Re-run all jobs

...

Summary

All jobs

✓ deploy

Run details

Usage

Workflow file

Triggered via push 10 hours ago

Status

Total duration

Artifacts

PeterHudcovic

 pushed

8812e08

main

Success

3m 5s

2

deploy.yml

on: push

✓ deploy

3m 1s

Run #3 was triggered by a push of commit 8812e08 to main, finished with status Success in 3 min 5 s and consisted of one job, deploy.

> ✓ Check for infrastructure changes	0s
> ✓ Run google-github-actions/auth@v2	1s
> ✓ Run docker/login-action@v3	3s
> ✓ Run docker/setup-buildx-action@v3	9s
> ✓ Build and push backend	55s
> ✓ Build and push frontend	20s
> ✓ Run google-github-actions/get-gke-credentials@v2	2s
> ✓ Run azure/setup-helm@v4	1s
> ✓ Check deployed version	7s
> ✓ helm upgrade	1m 4s
> ✓ Wait for rollout	3s
> ✓ Verify version through the public API	1s

Steps in order: check for infrastructure changes, WIF login, build and push backend and frontend, get GKE credentials, check the deployed version, helm upgrade (1 min 4 s), wait for rollout, verify the version through the public API.

>	✓	Check deployed version	7s
>	✓	helm upgrade	1m 4s
>	✓	Wait for rollout	3s
✓	✓	Verify version through the public API	1s
1	▶	Run set -euo pipefail	
34		Deployed version 8812e08538ae28974a856abbac3808fa93aa3f5c is live.	

The last step calls the public API and confirms that version 8812e08538ae28974a856abbac3808fa93aa3f5c is live, the same SHA as the image tags.

The screenshot displays the Sublime Merge application interface. On the left, the 'Locations' sidebar shows a tree of branches and worktrees. The 'Commits' tab is active, showing a vertical timeline of commits. The 'Files' tab shows the commit details for the selected commit. The 'Summary' tab shows the commit message and the pull request merge summary.

Commits:

- No changes
- Commit Changes
- Add display bezel and spacing to departure board (5)
- Contract: destination override map by IATA code (section 8)
- Polish departure board for live demo (#22)
- Merge pull request #20 from PeterHudcovic/claude/19-actions
- Merge pull request #18 from PeterHudcovic/claude/15-k8s
- Merge pull request #21 from PeterHudcovic/codex/17-frontend

Files:

- AGENTS.md
- CLAUDE.md
- contract.md

Summary:

- Commit Hash: 0f79adc2ed2948c1f86bdd33ef7c9596b4f1719a
- Tree: d41e300faf27a9095d09e9dd333f168684cc1270
- Author: Peter Hudcovič <124920481+PeterHudcovic@users.noreply.github.com>
- Committer: GitHub <noreply@github.com>
- Date: Sat, Oct 10, 2026, 12:10 AM
- Parents: 43ffaa28, 4d231d2a
- Signature: Invalid
- Stats: 3 files changed: -31 +432

Merge pull request #3 from PeterHudcovic/claude/2-contract

Fill docs/contract.md sections 1-8

AGENTS.md -1 +1

52 52 - Wait for the author's approval. Approval applies to the explained task within the agreed

53 53 scope; a change of scope needs new approval.

Sublime Merge on the author's machine. WORKTREES (3): flight-board on main, flight-board-claude on claude/24-destination-contract and flight-board-codex on codex/22-board-polish, so each agent worked in its own folder and branch. The graph shows the agents' branch commits and the pull request merges (#18, #20, #21) on main; this local main is 5 commits behind origin.

The screenshot shows the Google Cloud console interface. The left sidebar contains navigation links for Google Cloud, Cloud Storage, Overview, Buckets, Monitoring, Settings, Storage Intelligence, Advisor, Batch operations, Insights datasets, Configuration, Marketplace, and Release Notes. The main content area is titled 'Bucket details' for the bucket 'flight-board-prg-2610-tfstate'. It displays the bucket's location (europe-west3 (Frankfurt)), storage class (Standard), public access (Not public), and protection (Soft Delete, Object versioning). Below this, there are tabs for Objects, Configuration, Permissions, Protection, Lifecycle, Observability, and Operations. The 'Objects' tab is active, showing a 'Folder browser' on the left with a tree view containing 'flight-board-prg-2610-tfstate', 'bootstrap/', and 'main/'. On the right, there are buttons for 'Create folder', 'Upload', 'Transfer data', 'Create batch operations', and 'New'. Below these buttons is a table listing the objects in the bucket.

Name	Size	Type	Created	Storage class	Last modified	Custom conte
bootstrap/	—	Folder	—	—	—	—
main/	—	Folder	—	—	—	—

Terraform state for bootstrap and main infrastructure lives in a private, versioned Cloud Storage bucket in Frankfurt.

The screenshot shows the Google Cloud IAM & Admin console for the project "Flight Board". The left sidebar lists navigation options: IAM & Admin, Identity & Access, IAM, Service Accounts (selected), Groups, Privileged Access Ma..., Roles, Workload Identity Fed..., Workforce Identity Fe..., Principal Access Bou..., Managed Workload Id..., Resource Management, Organization Policies, Asset Inventory, Settings, Labels, Manage Resources, and Release Notes.

The main content area is titled "Service accounts" and includes buttons for "Create service account", "Delete", "Manage access", "Refresh", and "Learn". Below this, it says "Service accounts for project 'Flight Board'".

A service account represents a Google Cloud service identity, such as code running on Compute Engine VMs, App Engine apps, or systems running outside Google. [Learn more about service accounts.](#)

Organization policies can be used to secure service accounts and block risky service account features, such as automatic IAM Grants, key creation/upload, or the creation of service accounts entirely. [Learn more about service account organization policies.](#)

Below the text is a table of service accounts. The table has columns: Email, Status, Name, Description, Key ID, Key creation date, and Actions. There are four service accounts listed, all with a status of "Enabled" and "No keys".

Email	Status	Name	Description	Key ID	Key creation date	Actions
1051905559380-compute@developer.gserviceaccount.com	Enabled	Compute Engine default service account		No keys		
fb-deploy@flight-board-prg-2610.iam.gserviceaccount.com	Enabled	Flight Board deploy (GitHub Actions)		No keys		
fb-nodes@flight-board-prg-2610.iam.gserviceaccount.com	Enabled	Flight Board GKE nodes		No keys		
fb-tf-plan@flight-board-prg-2610.iam.gserviceaccount.com	Enabled	Flight Board		No keys		

Dedicated service accounts for Terraform plan, deploy and nodes, all with "No keys": GCP authentication uses WIF without long-lived service-account JSON keys.

Permissions for project "Flight Board"

These permissions affect this project and all of its resources. [Learn more](#)

☐ Include Google-provided role grants ?

View by principals

View by roles

+ Grant access

- Remove access

Filter fb- Enter property name or value

X ? |||

☐ Type	Principal ↑	Name	Role	Security insights ?
☐ 	fb-deploy@flight-board-prg-2610.iam.gserviceaccount.com	Flight Board deploy (GitHub Actions)	Kubernetes Engine Viewer	
☐ 	fb-nodes@flight-board-prg-2610.iam.gserviceaccount.com	Flight Board GKE nodes	Kubernetes Engine Default Node Service Account	
☐ 	fb-tf-plan@flight-board-prg-2610.iam.gserviceaccount.com	Flight Board terraform plan (GitHub Actions)	Viewer	

Project IAM filtered to fb-: fb-deploy has Kubernetes Engine Viewer, fb-nodes has Kubernetes Engine Default Node Service Account, fb-tf-plan has Viewer. Resource-level roles (Artifact Registry, state bucket) are not listed here; see the Terraform pages.

```
24 resource "google_project_iam_member" "tf_plan_viewer" {
25     project = var.project_id
26     role    = "roles/viewer"
27     member  = "serviceAccount:${google_service_account.tf_plan.email}"
28 }
29
30 # Includes container.clusters.connect for the DNS endpoint; write access inside the cluster
31 # comes only from the namespace RBAC Role (k8s/).
32 resource "google_project_iam_member" "deploy_container_viewer" {
33     project = var.project_id
34     role    = "roles/container.viewer"
35     member  = "serviceAccount:${google_service_account.deploy.email}"
36 }
37
38 resource "google_project_iam_member" "nodes_default" {
39     project = var.project_id
40     role    = "roles/container.defaultNodeServiceAccount"
41     member  = "serviceAccount:${google_service_account.nodes.email}"
42 }
```

infra/bootstrap/service_accounts.tf grants roles/viewer to fb-tf-plan, roles/container.viewer to fb-deploy (connect and read only; writes come from the namespace RBAC Role) and roles/container.defaultNodeServiceAccount to fb-nodes.

```
34     resource "google_artifact_registry_repository_iam_member" "deploy_writer" {
35         location    = google_artifact_registry_repository.images.location
36         repository  = google_artifact_registry_repository.images.name
37         role        = "roles/artifactregistry.writer"
38         member      = "serviceAccount:${google_service_account.deploy.email}"
39     }
40
41     resource "google_artifact_registry_repository_iam_member" "nodes_reader" {
42         location    = google_artifact_registry_repository.images.location
43         repository  = google_artifact_registry_repository.images.name
44         role        = "roles/artifactregistry.reader"
45         member      = "serviceAccount:${google_service_account.nodes.email}"
46     }
```

```
32     # terraform plan in GitHub Actions needs to read the state and write the lock file.
33     resource "google_storage_bucket_iam_member" "tf_plan_state" {
34         bucket = google_storage_bucket.tfstate.name
35         role   = "roles/storage.objectUser"
36         member = "serviceAccount:${google_service_account.tf_plan.email}"
37     }
```

infra/bootstrap/artifact_registry.tf: fb-deploy gets roles/artifactregistry.writer and fb-nodes roles/artifactregistry.reader on the flight-board repository only. infra/bootstrap/state_bucket.tf: fb-tf-plan gets roles/storage.objectUser on the Terraform state bucket only, to read state and write the lock file.

Security

Sandbox with gVisor	Disabled
Service account 	fb-nodes@flight-board-prg-2610.iam.gserviceaccount.com
GKE Metadata Server	Enabled
Integrity monitoring	Enabled
Secure boot	Enabled
Confidential GKE Nodes	Disabled

Metadata

Kubernetes labels 	project: flight-board
Labels 	goog-gke-node-pool-provisioning...: on-demand
Network tags 	flight-board-node
GCE instance metadata 	disable-legacy-endpoints: true

Live node pool flight-board-nodes: service account fb-nodes, GKE Metadata Server, Secure Boot and integrity monitoring enabled, network tag flight-board-node, legacy metadata endpoints disabled. This console page does not show the access scope.

```
99     resource "google_container_node_pool" "nodes" {
100         name          = "flight-board-nodes"
101         cluster       = google_container_cluster.main.id
102         location      = var.zone
103         node_count    = var.node_count
104
105     ...   management {
106         auto_repair   = true
107         auto_upgrade  = true
108     }
109
110     upgrade_settings {
111         max_surge      = 1
112         max_unavailable = 0
113     }
114
115     node_config {
116         machine_type   = var.machine_type
117         image_type     = "COS_CONTAINERD"
118         disk_type      = "pd-balanced"
119         disk_size_gb   = var.node_disk_size_gb
120         service_account = local.node_service_account
121         oauth_scopes    = ["https://www.googleapis.com/auth/cloud-platform"]
122         tags            = [local.node_tag]
```

infra/terraform/gke.tf sets the node pool service_account and oauth_scopes = cloud-platform; what the nodes can actually do is limited by the IAM roles of fb-nodes (default node role and Artifact Registry reader).

The screenshot shows the Google Cloud IAM & Admin console. The left sidebar lists navigation options under 'IAM & Admin', 'Identity & Access', and 'Resource Management'. The main content area is titled 'Edit github-actions provider' and shows a table of attribute mappings. The table has two columns: 'Google' and 'OIDC'. The 'Google' column lists attributes like 'attribute.repository_id' and 'attribute.workflow'. The 'OIDC' column lists corresponding attributes like 'assertion.repository_id' and 'assertion.workflow'. Below the table is a section for 'Attribute conditions' with a text input field containing the CEL expression 'assertion.repository_id == "1412231446"'. The right sidebar shows the 'Usage metric' for 'Workload Identity Federation', which includes a chart area with a warning icon and the text 'No data is available for the selected time frame'.

GitHub Actions signs in with short-lived OIDC tokens; the attribute condition accepts tokens only from this one repository ID.

google.subject ⓘ Supported keys: 'google.subject', 'google.groups', 'attribute.custom_attribute'.	assertion.sub ⓘ Value must be a CEL expression , for example , 'assertion.sub'.
Google 2 * attribute.deploy ⓘ Supported keys: 'google.subject', 'google.groups', 'attribute.custom_attribute'.	OIDC 2 * (assertion.ref == 'refs/heads/main' && ⓘ ⓘ Value must be a CEL expression , for example , 'assertion.sub'.
Google 3 * attribute.event_name ⓘ Supported keys: 'google.subject', 'google.groups', 'attribute.custom_attribute'.	OIDC 3 * assertion.event_name ⓘ Value must be a CEL expression , for example , 'assertion.sub'.
Google 4 * attribute.ref ⓘ Supported keys: 'google.subject', 'google.groups', 'attribute.custom_attribute'.	OIDC 4 * assertion.ref ⓘ Value must be a CEL expression , for example , 'assertion.sub'.
Google 5 * attribute.repository_id ⓘ Supported keys: 'google.subject', 'google.groups', 'attribute.custom_attribute'.	OIDC 5 * assertion.repository_id ⓘ Value must be a CEL expression , for example , 'assertion.sub'.
Google 6 * attribute.workflow ⓘ Supported keys: 'google.subject', 'google.groups', 'attribute.custom_attribute'.	OIDC 6 * assertion.workflow ⓘ Value must be a CEL expression , for example , 'assertion.sub'.

Attribute conditions

Restrict authentication to a subset of identities. By default, all identities belonging to providers in this pool can authenticate. [Learn more.](#) ⓘ

Condition CEL ⓘ
assertion.repository_id == "1412231446" ⓘ

Use a [CEL expression](#), ⓘ for example, 'admins' in google.groups'.

Provider github-actions maps google.subject, attribute.deploy, attribute.event_name, attribute.ref, attribute.repository_id and attribute.workflow from the GitHub token. The condition assertion.repository_id == "1412231446" rejects tokens from any other repository. Viewed in the edit form; nothing was saved.

To access resources, pool identities must be granted access to a service account. Once added, these identities will have permission to access any Google Cloud services that the service account can access. [Learn more.](#) 

The table below displays the connected service accounts within the current project.

Service account	Client library config	
▼ fb-tf-plan	 Download	
attribute.repository_id="1412231446"		
▼ fb-deploy	 Download	
attribute.deploy="true"		

Connected service accounts of pool github: fb-tf-plan can be used by any identity with attribute.repository_id = 1412231446; fb-deploy only by identities with attribute.deploy = true.

```
18 # Only tokens from this repository are accepted at all.
19 attribute_condition = "assertion.repository_id == \"${var.github_repository_id}\""
20
21 attribute_mapping = {
22   "google.subject"      = "assertion.sub"
23   "attribute.repository_id" = "assertion.repository_id"
24   "attribute.ref"        = "assertion.ref"
25   "attribute.workflow"   = "assertion.workflow"
26   "attribute.event_name" = "assertion.event_name"
27   # "true" only for the Deploy workflow on main, triggered by push or manual dispatch.
28   "attribute.deploy" = "(assertion.ref == 'refs/heads/main' && assertion.workflow == 'Deploy' && (assertion.event_name == 'push' || assertion.event_name == 'workflow_dispatch')) ? 'true' : 'false'"
29 }
30 }
```

infra/bootstrap/wif.tf: the provider accepts only tokens from this repository ID. attribute.deploy is "true" only when ref is refs/heads/main, workflow is Deploy and the event is push or workflow_dispatch.

```
36 # fb-tf-plan: any workflow run of this repository.
37 resource "google_service_account_iam_member" "tf_plan_wif" {
38   service_account_id = google_service_account.tf_plan.name
39   role                = "roles/iam.workloadIdentityUser"
40   member              = "principalSet://iam.googleapis.com/${local.wif_pool}/attribute.repository_id/${var.github_repository_id}"
41 }
42
43 # fb-deploy: only the Deploy workflow on main.
44 resource "google_service_account_iam_member" "deploy_wif" {
45   service_account_id = google_service_account.deploy.name
46   role                = "roles/iam.workloadIdentityUser"
47   member              = "principalSet://iam.googleapis.com/${local.wif_pool}/attribute.deploy/true"
48 }
```

Both bindings grant roles/iam.workloadIdentityUser: fb-tf-plan to principalSet .../attribute.repository_id/<repo id> (any workflow of this repository), fb-deploy to principalSet .../attribute.deploy/true (only Deploy on main).

```
1  # Build, push and deploy main to GKE (spec chapter 13, Appendix A2/A7).
2  # The workflow name "Deploy" is part of the WIF condition for fb-deploy (infra/bootstrap/wif.tf).
3  name: Deploy
4
5  on:
6    push:
7      branches: [main]
8    workflow_dispatch:
9
10 permissions:
11   contents: read
12   id-token: write
13
14 # One deployment at a time; a running deployment is never cancelled.
15 concurrency:
16   group: deploy-production
17   cancel-in-progress: false
```

```
56     - id: auth
57       if: steps.gate.outputs.skip != 'true'
58       uses: google-github-actions/auth@v2
59       with:
60         workload_identity_provider: ${vars.GCP_WORKLOAD_IDENTITY_PROVIDER}
61         service_account: fb-deploy@flight-board-prg-2610.iam.gserviceaccount.com
62         token_format: access_token
```

.github/workflows/deploy.yml is named Deploy (part of the WIF rule), runs on push to main and manual dispatch, and requests id-token: write. Its auth step uses the provider from repository variable GCP_WORKLOAD_IDENTITY_PROVIDER and impersonates fb-deploy.

```
142     # fb-tf-plan has no billing permission for the budget (docs/contract.md section 5).
143     terraform-plan:
144       needs: [ownership, terraform]
145       # No plan for PRs from forks: they get no OIDC token for this repository's provider.
146       if: >-
147         needs.ownership.outputs.terraform == 'true' &&
148         github.event.pull_request.head.repo.full_name == github.repository
149       runs-on: ubuntu-latest
150       permissions:
151         contents: read
152         id-token: write
153       steps:
154         - uses: actions/checkout@v4
155
156         - uses: google-github-actions/auth@v2
157           with:
158             workload_identity_provider: ${vars.GCP_WORKLOAD_IDENTITY_PROVIDER}
159             service_account: fb-tf-plan@flight-board-prg-2610.iam.gserviceaccount.com
```

.github/workflows/ci.yml, job terraform-plan: runs only for pull requests from this repository (forks get no OIDC token), requests id-token: write and impersonates fb-tf-plan.

```
1  # Namespace rights of the GitHub Actions deploy account (docs/contract.md section 5).
2  # The GCP role roles/container.viewer only allows connecting and reading; all writes come from here.
3  # No rights on MongoDBCommunity or the operator: MongoDB is managed only by the author.
4  apiVersion: rbac.authorization.k8s.io/v1
5  kind: Role
6  metadata:
7    name: flight-board-deployer
8    namespace: flight-board
9    labels:
10     app.kubernetes.io/part-of: flight-board
11  rules:
12  - apiGroups: [""]
13    # secrets: Helm stores its release history in Secrets.
14    resources: ["services", "configmaps", "secrets", "serviceaccounts"]
15    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
16  - apiGroups: [""]
17    resources: ["pods", "events"]
18    verbs: ["get", "list", "watch"]
19  - apiGroups: ["apps"]
20    resources: ["deployments"]
21    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
22  - apiGroups: ["apps"]
23    resources: ["replicasets"]
24    verbs: ["get", "list", "watch"]
25  - apiGroups: ["policy"]
26    resources: ["poddisruptionbudgets"]
27    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
28  - apiGroups: ["networking.k8s.io"]
29    resources: ["ingresses"]
30    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
```

k8s/rbac/deployer.yaml: Role flight-board-deployer in namespace flight-board can manage services, configmaps, secrets, serviceaccounts, deployments, PDBs and ingresses, and only read pods, events and replicasets. It has no rights on MongoDB or the operator.

```
31   - apiGroups: ["cloud.google.com"]
32     resources: ["backendconfigs"]
33     verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
34   ---
35   apiVersion: rbac.authorization.k8s.io/v1
36   kind: RoleBinding
37   metadata:
38     name: flight-board-deployer
39     namespace: flight-board
40     labels:
41       app.kubernetes.io/part-of: flight-board
42   roleRef:
43     apiGroup: rbac.authorization.k8s.io
44     kind: Role
45     name: flight-board-deployer
46   subjects:
47   - apiGroup: rbac.authorization.k8s.io
48     kind: User
49     name: fb-deploy@flight-board-prg-2610.iam.gserviceaccount.com
```

The Role also manages backendconfigs. RoleBinding flight-board-deployer gives the Role to user fb-deploy@flight-board-prg-2610.iam.gserviceaccount.com, the identity GitHub Actions gets through WIF.

DNS records for peterhudcovic.tech

DNS Setup: Full [DNS documentation](#)

Manage how the Internet finds your web content, verifies services, and routes traffic.

other record, not part of this project

Q Search DNS Records

Filters

Display options

Import

Export

+ Add record

You have used 6 of 200 available DNS records in this domain.

☐	ⓘ Name ⬆ ⬆	Type ⬆ ⬆	Content ⬆ ⬆	Proxy status ⬆ ⬆	TTL ⬆ ⬆	Tags	Comment	Details	
	other record, not part of this project								
☐	flights.peterhudcovic.tech	A	35.190.55.104	 Proxied	Auto	—	—	—	Edit
	other record, not part of this project								

Showing 1 - 6 of 6

DNS record flights.peterhudcovic.tech (type A) points to the load balancer IP 35.190.55.104 and is Proxied (orange cloud), so visitors connect to Cloudflare, not directly to Google Cloud. Other records of the domain are not part of this project and are covered in grey.

SSL/TLS

[Documentation](#)[Configure](#)

Choose how Cloudflare encrypts traffic between your visitors and Cloudflare, and between Cloudflare and your origin server, to prevent data theft and other tampering.

SSL/TLS encryption

Current encryption mode: **Full**



Automatic mode enabled · 70 days ago

Next automatic scan: Oct 31

Traffic Served Over TLS Last 24 hours

None (not secure)

51

TLSv1.2

47

TLSv1.3

3.47k



Zone-wide SSL/TLS encryption mode for peterhudcovic.tech is Full (automatic mode). Full does not validate the origin certificate, so the flights hostname is raised to Full (strict) by the Configuration Rule on the next page. The traffic counters cover the whole domain, not only this project.

Configuration Rules 2 active

Create rule

Order	Name	Match against	Action	Status
other rule, not part of this project				
2	<u>flights strict SSL</u>	Hostname equals flights.peterhudcovic.tech	SSL	Active ...

Configuration Rule flights strict SSL is active and matches Hostname equals flights.peterhudcovic.tech; its action is an SSL setting. The other rule of the domain is not part of this project and is covered in grey.

Edit Configuration Rule

Rule name (required)

flights strict SSL

Give your rule a descriptive name.

If incoming requests match...

☒ **Custom filter expression**
Only apply the rule to requests matching the custom filter expression

☐ **All incoming requests**
Apply the rule to all requests

When incoming requests match...

Field	Operator	Value
Hostname X	equals ⌵	flights.peterhudcovic.tech e.g. example.com

And Or

Expression Preview [Edit expression](#)

(http.host eq "flights.peterhudcovic.tech")

SSL (optional)

Configure SSL settings available in SSL/TLS > Overview tab.

Select SSL/TLS encryption mode

Strict

Condition: custom filter expression (http.host eq "flights.peterhudcovic.tech"). Setting: SSL/TLS encryption mode Strict. So only this hostname uses Full (strict) and Cloudflare validates the Origin certificate on the Google load balancer, while the rest of the zone stays on Full. Viewed in the edit form; nothing was saved.

71. Free trial credit and month to date

Cost

Group by (Service) ▾

Time range by charge period (Current month) ▾

Projects (1 of 3) ▾

Savings (All) ▾

Show more ▾

You saved -Kč22.38 during this period. To view recommendations on how to save more, [visit the FinOps hub](#)

Free trial credits ⓘ CZK2,401.42 remaining out of CZK6,373.25	October 1 – 10, 2026 CZK0.00 includes -CZK22.38 in savings	October 1 – 31, 2026 (forecasted) ⓘ Not enough historical data to project cost
--	---	--

Billing report filtered to project Flight Board: 1 to 10 October 2026 net cost CZK 0.00 because CZK 22.38 of usage was covered by free trial credit; CZK 2,401.42 of CZK 6,373.25 trial credit remains on the billing account. Billing data can lag by up to a day.

72. Cost by service

Cost

Service	Usage cost ?	Negotiated savings ?	Savings programs ?	Other savings ?	↓ Subtotal	% Change ?
● Artifact Registry	Kč0.00	Kč0.00	—	Kč0.00	Kč0.00	New
■ Kubernetes Engine	Kč4.89	Kč0.00	—	-Kč4.89	Kč0.00	New
◆ Networking	Kč4.65	Kč0.00	—	-Kč4.65	Kč0.00	New
▼ Compute Engine	Kč12.84	Kč0.00	—	-Kč12.84	Kč0.00	New
					Subtotal	Kč0.00
					Tax ?	—
					Filtered total ?	Kč0.00

Usage cost so far: Compute Engine CZK 12.84, Kubernetes Engine CZK 4.89, Networking CZK 4.65, Artifact Registry CZK 0.00; each is fully offset by credits (Other savings), subtotal CZK 0.00.

☐	Budget name ↑	Budget period	Budget type	Applies to	Trigger alerts at	Spend and budget amount ?	Spend cap status ?
☐	flight-board-prg-2610	Monthly	Specified amount	Proje... ▼	50%, 90%, and 100%	Kč0.00 / Kč1,000.00 Includes -Kč22.38 savings	Not applicable

Monthly budget flight-board-prg-2610 of CZK 1,000 for the project with alerts at 50 %, 90 % and 100 %; current spend CZK 0.00 after CZK 22.38 savings. It is an alert budget, not a spend cap.

The screenshot shows the GitHub interface for the 'flight-board' repository. The repository is public and has 0 stars, 0 forks, and 0 watches. The main branch is 'main'. The repository contains several files and folders, including .github, app, docs, infra, k8s, .gitattributes, .gitignore, AGENTS.md, CLAUDE.md, and README.md. The repository is described as 'Airport departure board on GKE with Terraform'. The repository has 36 commits and a recent merge pull request #23 from PeterHudcovic/codex/22-board-polish.

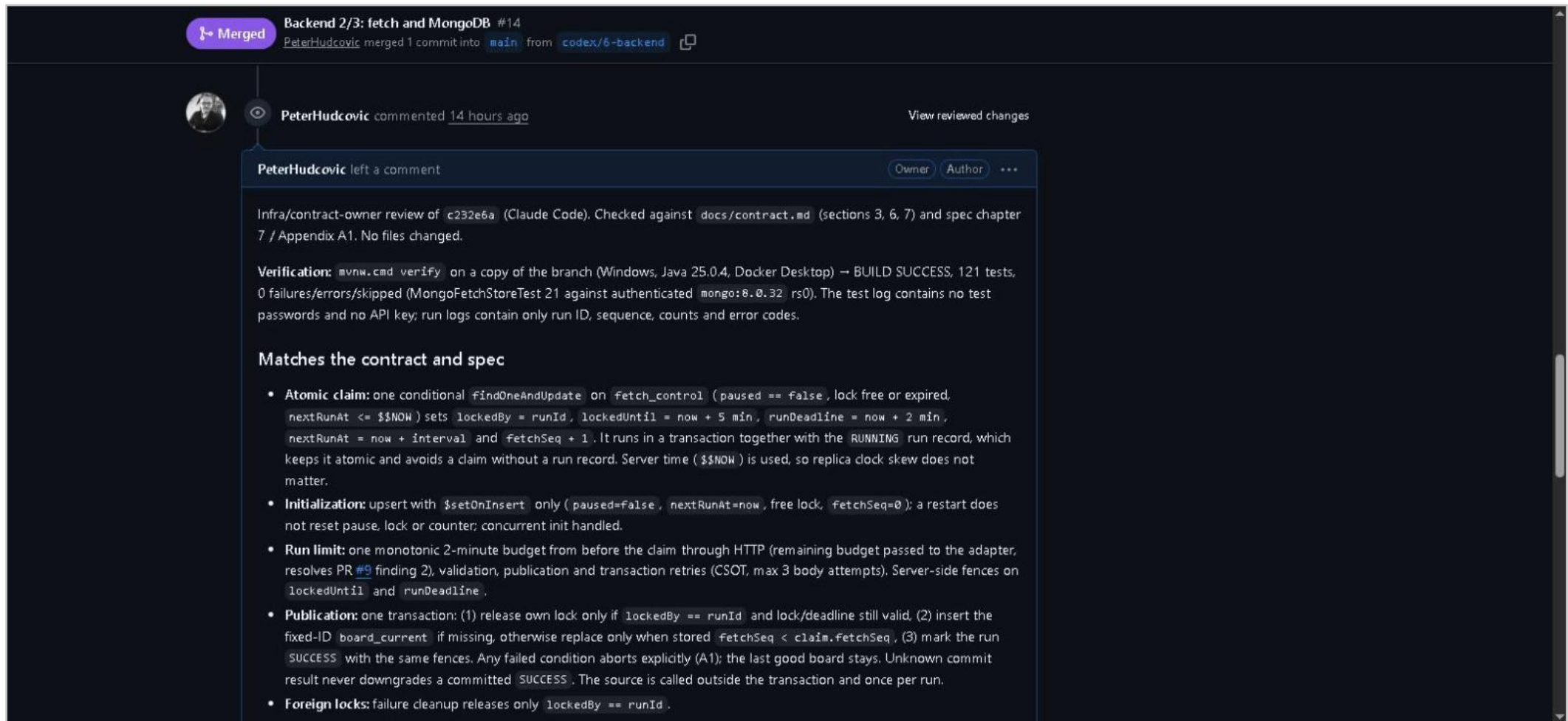
File/Folder	Description	Time
.github	deploy.yml: fail closed when the deployed version cannot be r...	12 hours ago
app	Add display bezel and spacing to departure board	9 hours ago
docs	Contract: destination override map by IATA code (section 8)	9 hours ago
infra	Merge pull request #11 from PeterHudcovic/claude/10-terraf...	14 hours ago
k8s	k8s: set the board domain flights.peterhudcovic.tech	13 hours ago
.gitattributes	Stage 0: add root repository files	19 hours ago
.gitignore	Address review: branch workflow, mvnw in PowerShell, gitigno...	19 hours ago
AGENTS.md	Fill docs/contract.md sections 1-8	16 hours ago
CLAUDE.md	Fill docs/contract.md sections 1-8	16 hours ago
README.md	Add GitHub Actions ci.yml and deploy.yml, API shape in contr...	13 hours ago

The public repository holds the application, Terraform, Kubernetes manifests, workflows and the written contract between the two AI agents.

The screenshot shows the GitHub interface for the 'flight-board' repository. The 'Pull requests' tab is selected, displaying a list of 13 merged pull requests. The left sidebar contains filters for 'Authored by me', 'Assigned to me', 'Involves me', and 'Review requests'. The main area shows a list of pull requests with details such as title, number, author, merge time, and status. A search bar at the top right allows filtering by 'is:pr is:dosed'. A 'New pull request' button is visible in the top right corner.

Open	Closed	Author	Label	Projects	Milestones	Reviews	Assignee	Newest
0	13							
☐	☒							
Contract: destination override map (section 8)								
#25 · by PeterHudcovic was merged 9 hours ago (Owner) · ✓ 5/5								
☐	☒							
Polish departure board for live demo								
#23 · by PeterHudcovic was merged 9 hours ago (Owner) · 5 tasks done · ✓ 5/5								
☐	☒							
Frontend: departure board, cache and nginx								
#21 · by PeterHudcovic was merged 13 hours ago (Owner) · 5 tasks done								
☐	☒							
GitHub Actions: ci.yml and deploy.yml								
#20 · by PeterHudcovic was merged 12 hours ago (Owner) · ✓ 5/5								
☐	☒							
Kubernetes manifests and Helm chart (k8s/)								
#18 · by PeterHudcovic was merged 13 hours ago (Owner)								
☐	☒							
Backend 3/3: API, admin and container								
#16 · by PeterHudcovic was merged 13 hours ago (Owner) · 5 tasks done								
☐	☒							
Backend 2/3: fetch and MongoDB								
#14 · by PeterHudcovic was merged 14 hours ago (Owner) · 5 tasks done								

Every change went through a pull request: 13 merged PRs, each reviewed by the other agent and merged by me.



The screenshot shows a GitHub pull request interface. At the top, a purple banner indicates a merge: "Merged Backend 2/3: fetch and MongoDB #14". Below this, a comment by PeterHudcovic is shown, dated "14 hours ago". The comment text reads: "Infra/contract-owner review of c232e6a (Claude Code). Checked against docs/contract.md (sections 3, 6, 7) and spec chapter 7 / Appendix A1. No files changed." Below the text, there is a "Verification" section: "Verification: mvnw.cmd verify on a copy of the branch (Windows, Java 25.0.4, Docker Desktop) - BUILD SUCCESS, 121 tests, 0 failures/errors/skipped (MongoFetchStoreTest 21 against authenticated mongo:8.0.32 rs0). The test log contains no test passwords and no API key; run logs contain only run ID, sequence, counts and error codes." A section titled "Matches the contract and spec" follows, containing a bulleted list of review points: "Atomic claim", "Initialization", "Run limit", "Publication", and "Foreign locks".

Merged Backend 2/3: fetch and MongoDB #14
PeterHudcovic merged 1 commit into main from codex/6-backend

PeterHudcovic commented 14 hours ago [View reviewed changes](#)

PeterHudcovic left a comment Owner Author ...

Infra/contract-owner review of c232e6a (Claude Code). Checked against docs/contract.md (sections 3, 6, 7) and spec chapter 7 / Appendix A1. No files changed.

Verification: mvnw.cmd verify on a copy of the branch (Windows, Java 25.0.4, Docker Desktop) - BUILD SUCCESS, 121 tests, 0 failures/errors/skipped (MongoFetchStoreTest 21 against authenticated mongo:8.0.32 rs0). The test log contains no test passwords and no API key; run logs contain only run ID, sequence, counts and error codes.

Matches the contract and spec

- **Atomic claim:** one conditional findOneAndUpdate on fetch_control (paused == false, lock free or expired, nextRunAt <= \$NOW) sets lockedBy = runId, lockedUntil = now + 5 min, runDeadline = now + 2 min, nextRunAt = now + interval and fetchSeq + 1. It runs in a transaction together with the RUNNING run record, which keeps it atomic and avoids a claim without a run record. Server time (\$NOW) is used, so replica clock skew does not matter.
- **Initialization:** upsert with \$setOnInsert only (paused=false, nextRunAt=now, free lock, fetchSeq=0); a restart does not reset pause, lock or counter; concurrent init handled.
- **Run limit:** one monotonic 2-minute budget from before the claim through HTTP (remaining budget passed to the adapter, resolves PR #9 finding 2), validation, publication and transaction retries (CSOT, max 3 body attempts). Server-side fences on lockedUntil and runDeadline.
- **Publication:** one transaction: (1) release own lock only if lockedBy == runId and lock/deadline still valid, (2) insert the fixed-ID board_current if missing, otherwise replace only when stored fetchSeq < claim.fetchSeq, (3) mark the run SUCCESS with the same fences. Any failed condition aborts explicitly (A1); the last good board stays. Unknown commit result never downgrades a committed SUCCESS. The source is called outside the transaction and once per run.
- **Foreign locks:** failure cleanup releases only lockedBy == runId.

Example review: Claude Code (infrastructure owner) checked Codex's MongoDB coordination code against the contract and re-ran all 121 tests before approving.

The screenshot shows the Google Cloud Logs Explorer interface. At the top, the 'Project logs' dropdown is set to 'Flight Board'. The search bar contains the query: `resource.type="k8s_container"`, `resource.labels.namespace_name="flight-board"`, and `resource.labels.pod_name=~"flight-board-backend"`. The 'Kubernetes Container +1' filter is applied. The 'Timeline' view shows a horizontal timeline with two blue bars indicating log events at 15:17 and 15:47. Below the timeline, the '4 results' section displays a table of log entries.

Severity	Time	Summary
INFO	2026-10-10 15:17:27.395	[flight-board-backend] [virtual-2115] com.flightboard.fetch.FetchCoordinator : Run 43592244-a9ba-48df-9c61-65cca7b283cd c-
INFO	2026-10-10 15:17:27.656	[flight-board-backend] [virtual-2115] com.flightboard.fetch.FetchCoordinator : Run 43592244-a9ba-48df-9c61-65cca7b283cd p-
INFO	2026-10-10 15:47:27.438	[flight-board-backend] [virtual-2236] com.flightboard.fetch.FetchCoordinator : Run 7ece142d-e1ed-4e45-8f92-245284627437 c-
INFO	2026-10-10 15:47:27.728	[flight-board-backend] [virtual-2236] com.flightboard.fetch.FetchCoordinator : Run 7ece142d-e1ed-4e45-8f92-245284627437 p-

Backend logs in Cloud Logging show the scheduled fetch every 30 minutes (15:17, 15:47), each with its own run ID for troubleshooting.